

QCon
全球软件开发大会

从烟囱到积木：基于openYuanrong的AI Infra实践

吴杰

华为 系统软件架构师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01

Agentic AI负载对分布式系统带来的变化和挑战

02

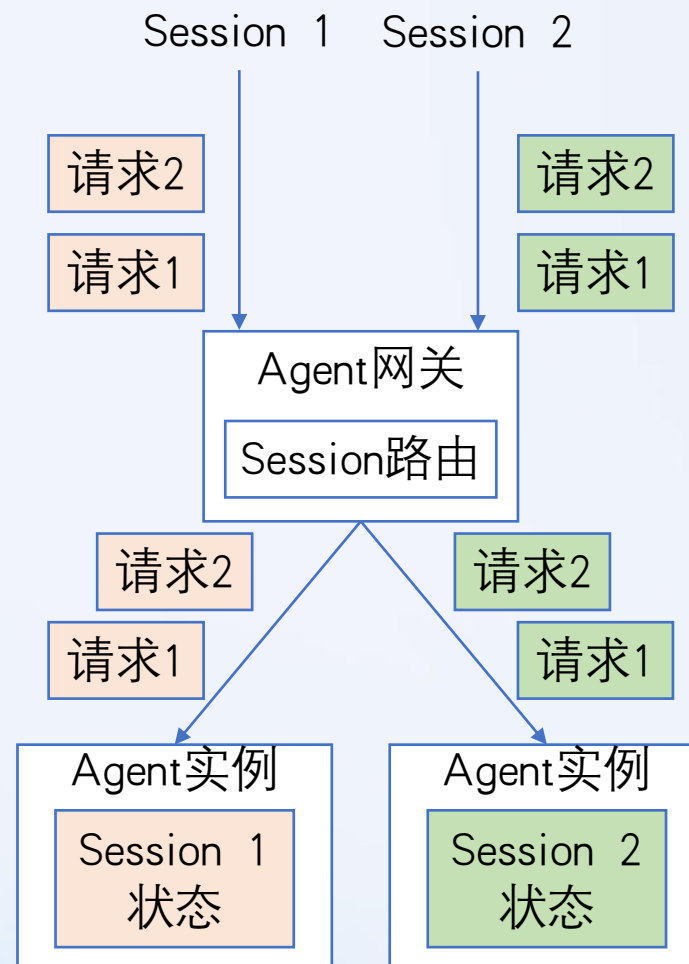
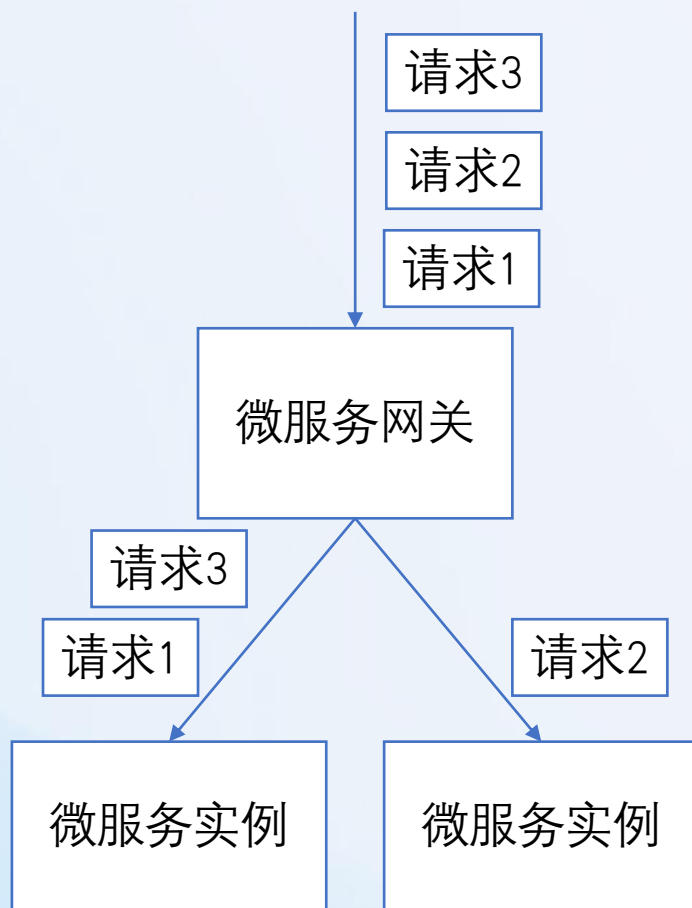
openYuanrong Serverless分布式计算引擎简介

03

openYuanrong面向Agentic AI负载的关键技术实践

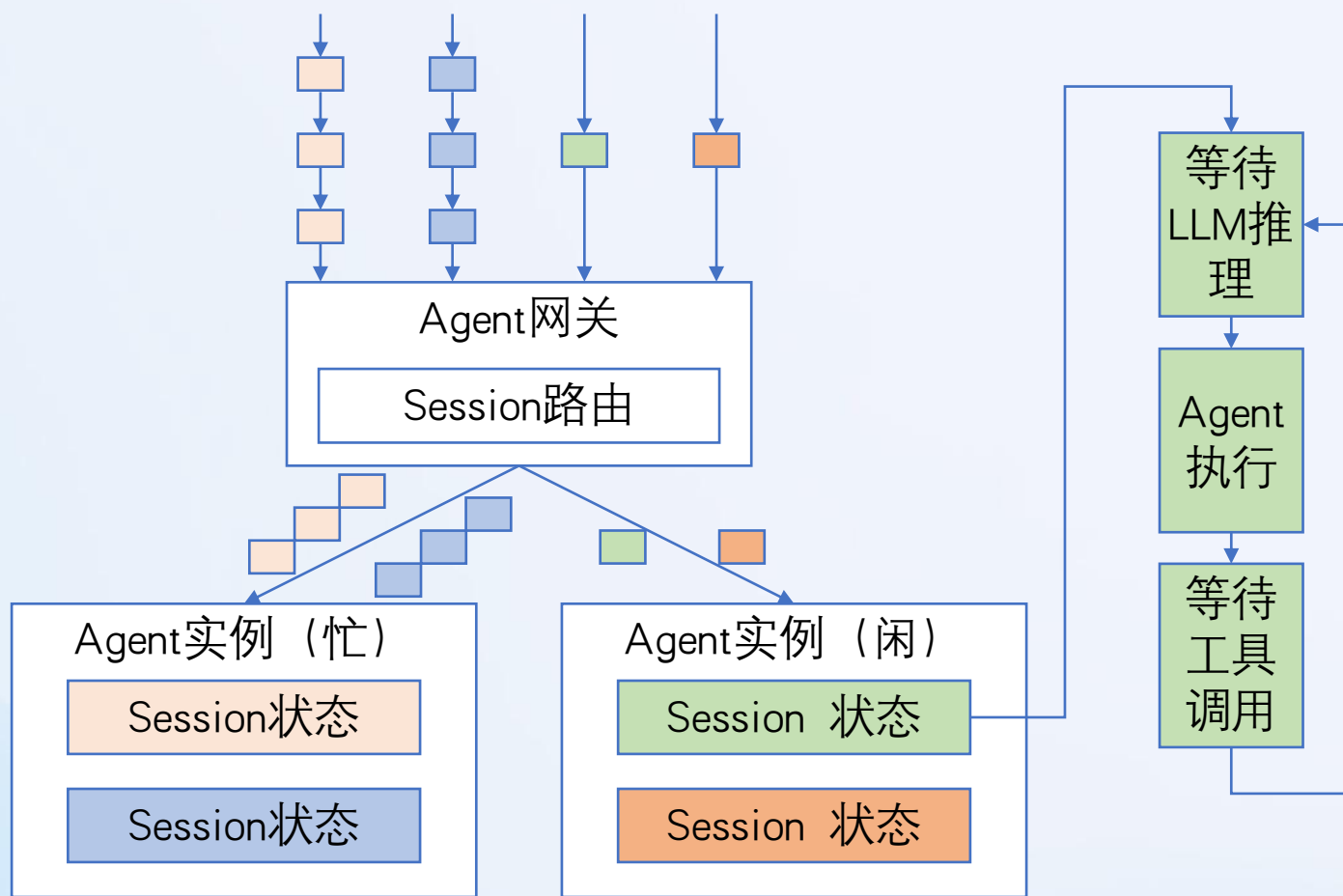


Agent服务的变化：从毫秒级的无状态请求到小时级的有状态 session



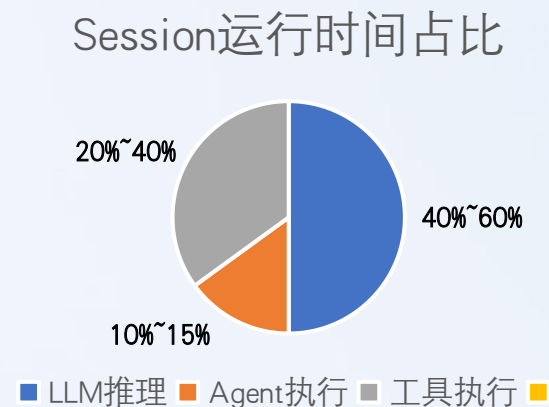
session状态:
1. 历史上下文
2. 内部任务状态
3. ...

Agent服务的挑战：提升长程session并发度和资源利用率



问题1. 固定session并发数导致负载不均衡

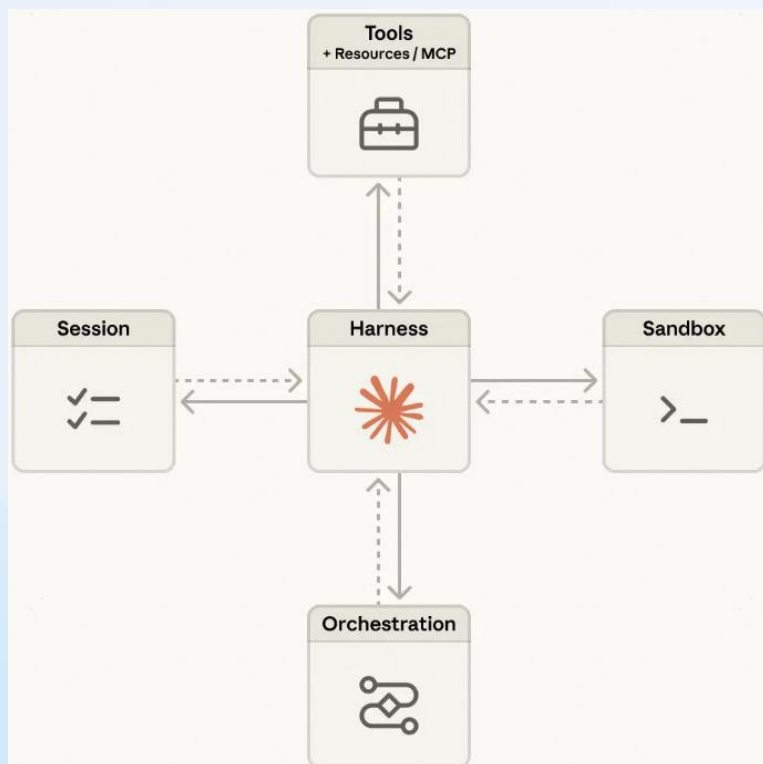
问题2. session运行过程中的线程cpu负载低



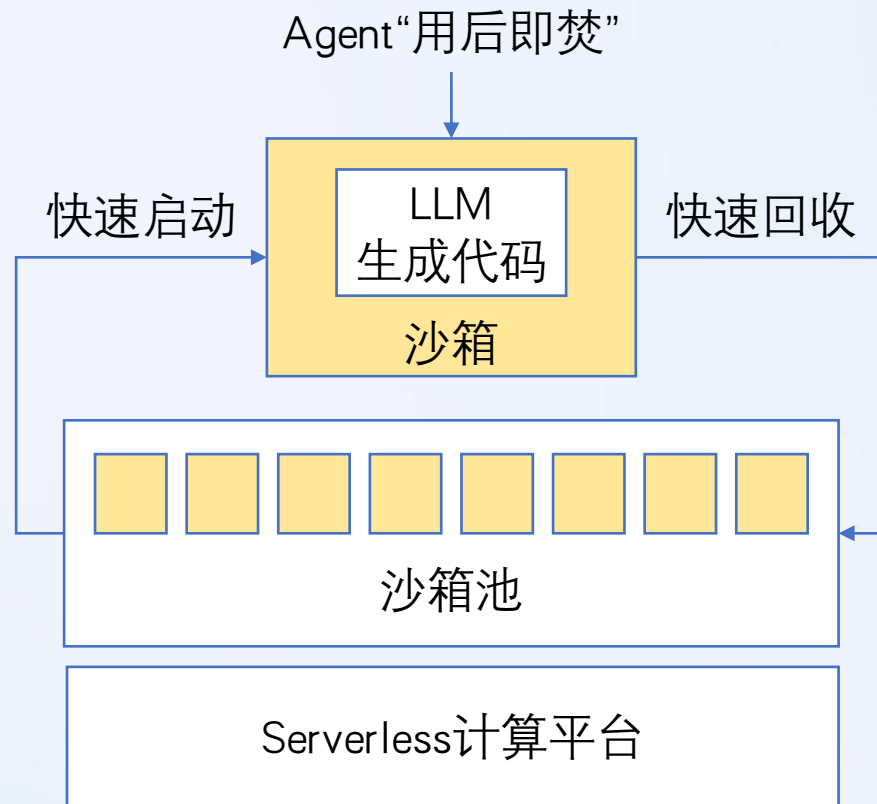
沙箱系统的挑战：启动+回收速度提升一个数量级以上

抽象成“函数调用”的沙箱成为AI Agent基础设施

装载动态生成代码的一次性沙箱，快速启动+快速回收



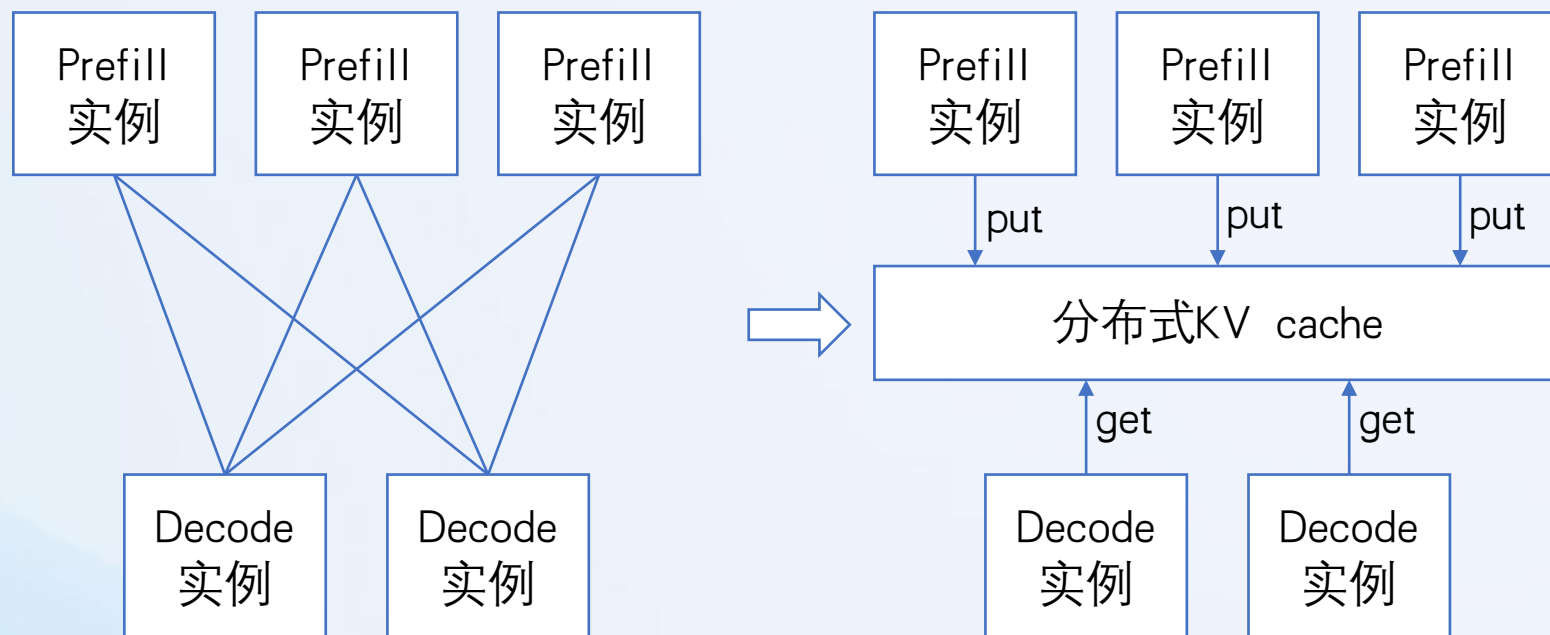
```
provision({}resources)
execute(name, input)
-> String
```



图片来源：<https://www.anthropic.com/engineering/managed-agents>

大模型推理服务：回顾分布式KV cache系统

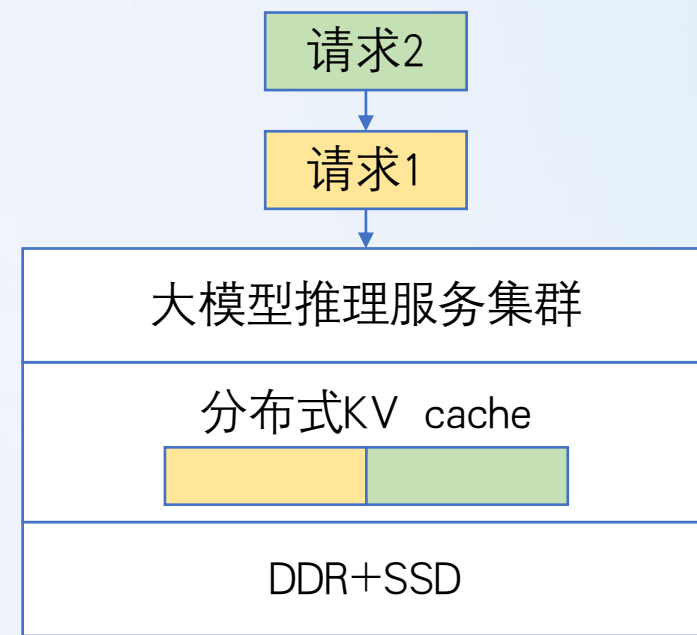
分离式推理架构下的KV cache点对点异步传输



P/D模型实例之间无法基于同步集合通信编程协调KV cache传输

分布式KV cache提供异步的put/get接口实现KV cache传输

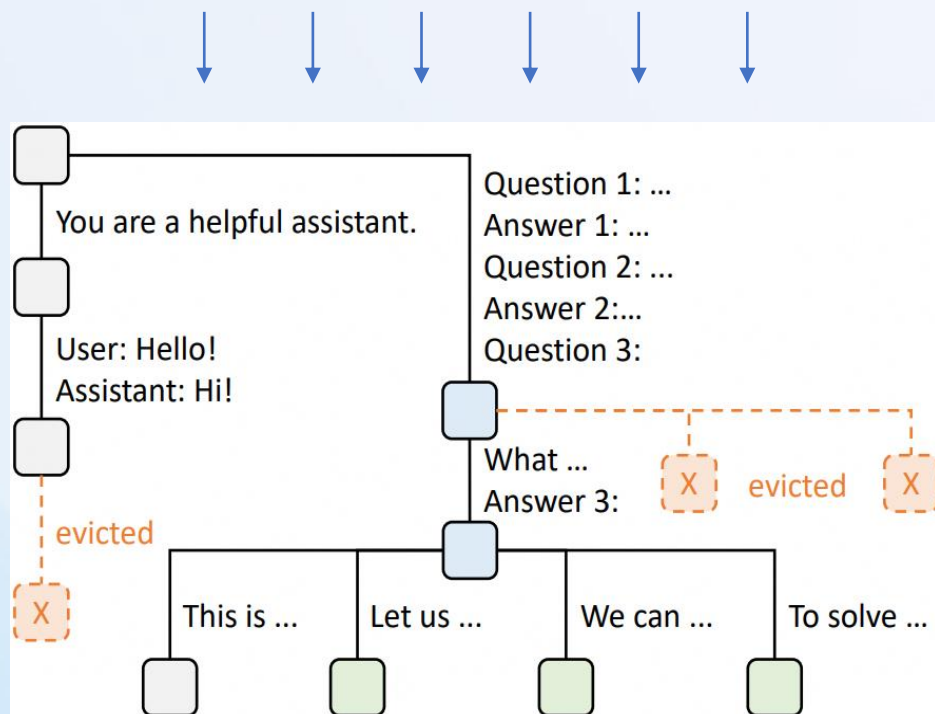
历史缓存加速



分布式KV cache池化管理集群存储介质实现历史缓存加速

分布式KV cache的变化：从全局前缀共享到会话间时分共享

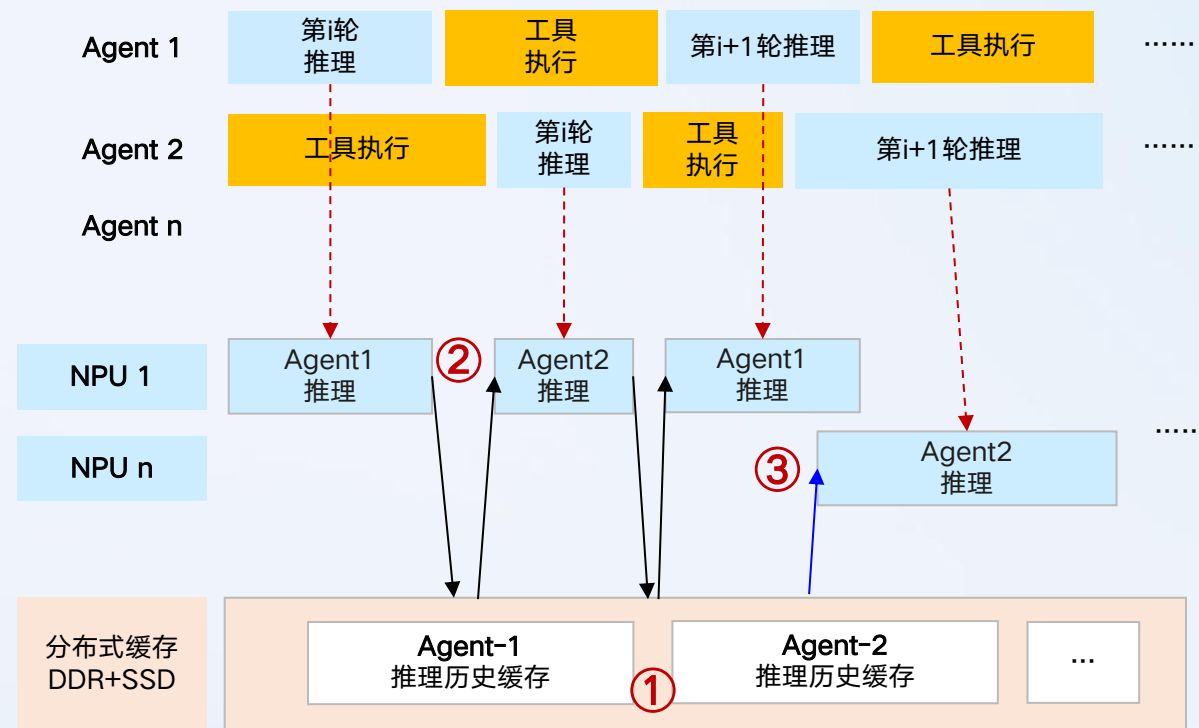
基于前缀树全局管理
不同请求的历史KV cache



图片引用

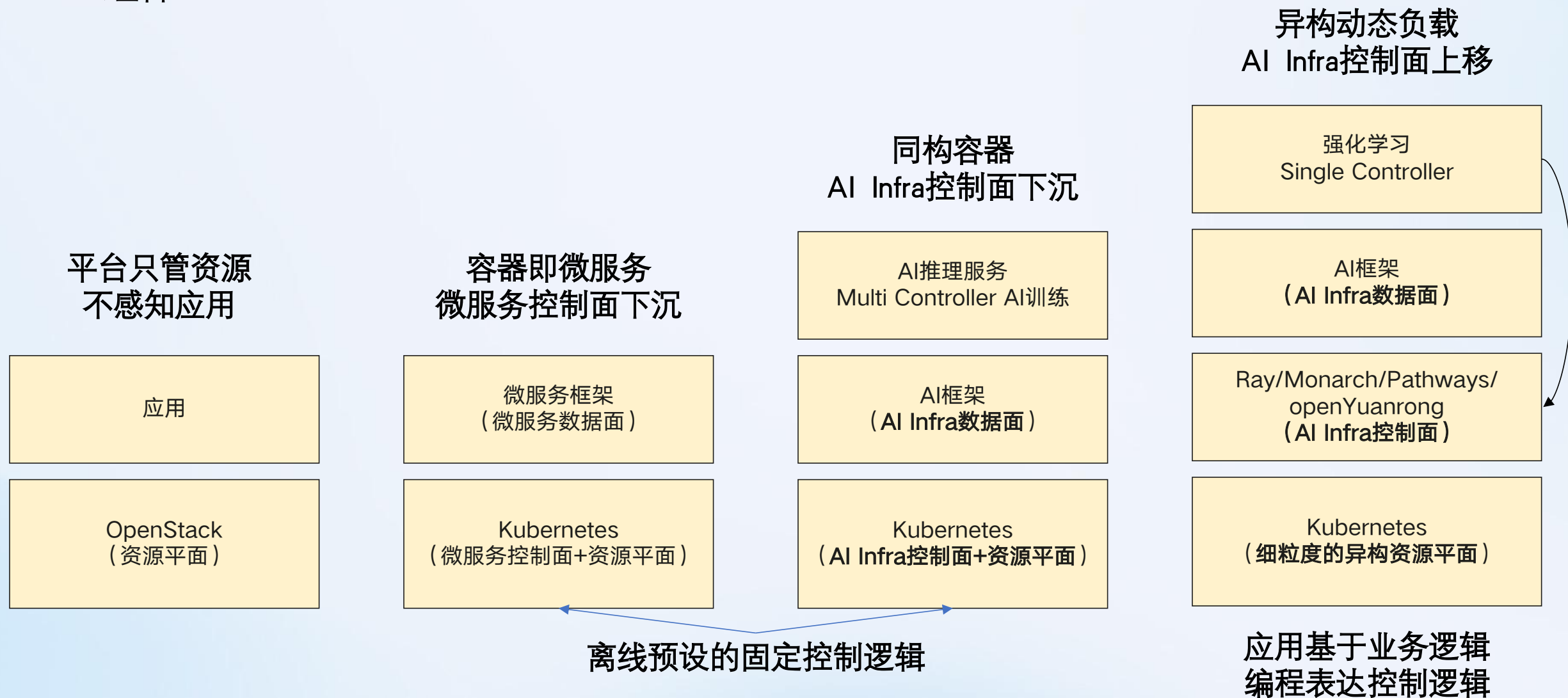
<https://arxiv.org/abs/2312.07104>

Agent会话内共享历史KV cache，会话间不共享





强化学习推动AI Infra控制面的架构演进, Ray/openYuanrong等分布式框架成为基础组件

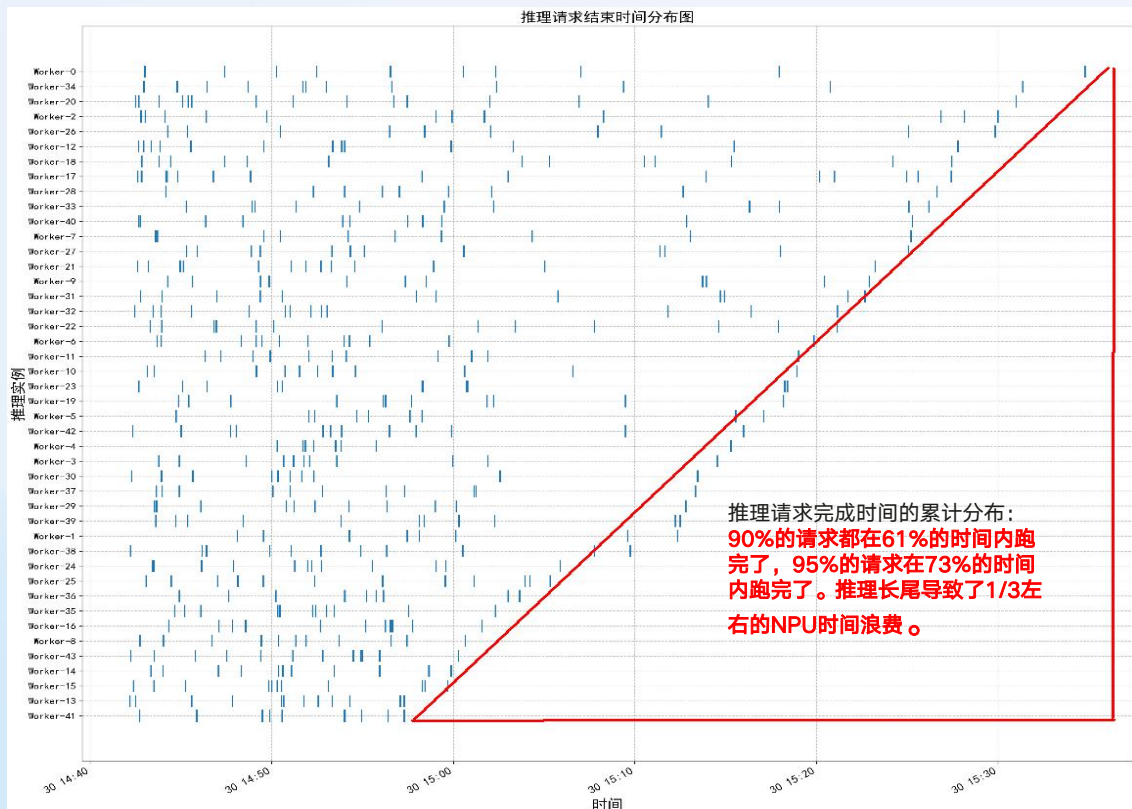




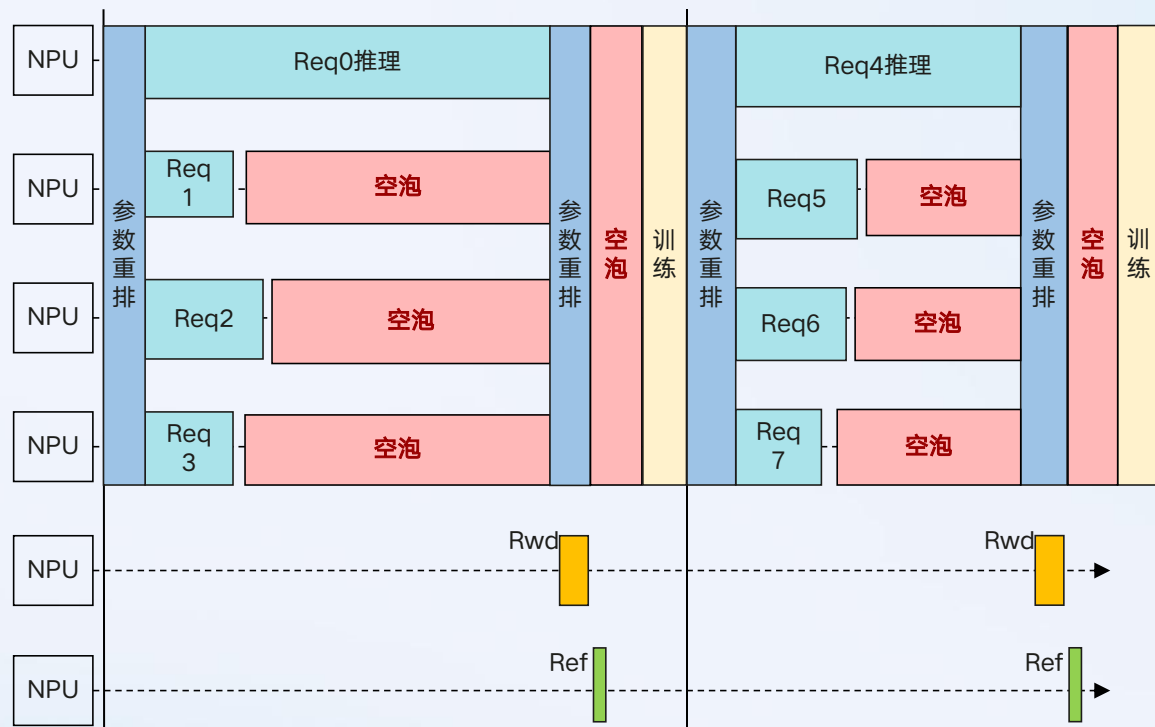
强化学习系统的变化：Agentic RL推动RL系统从训推同步共卡到训推异步分离

训推共卡架构下，Agentic model长短采样任务导致NPU资源空泡

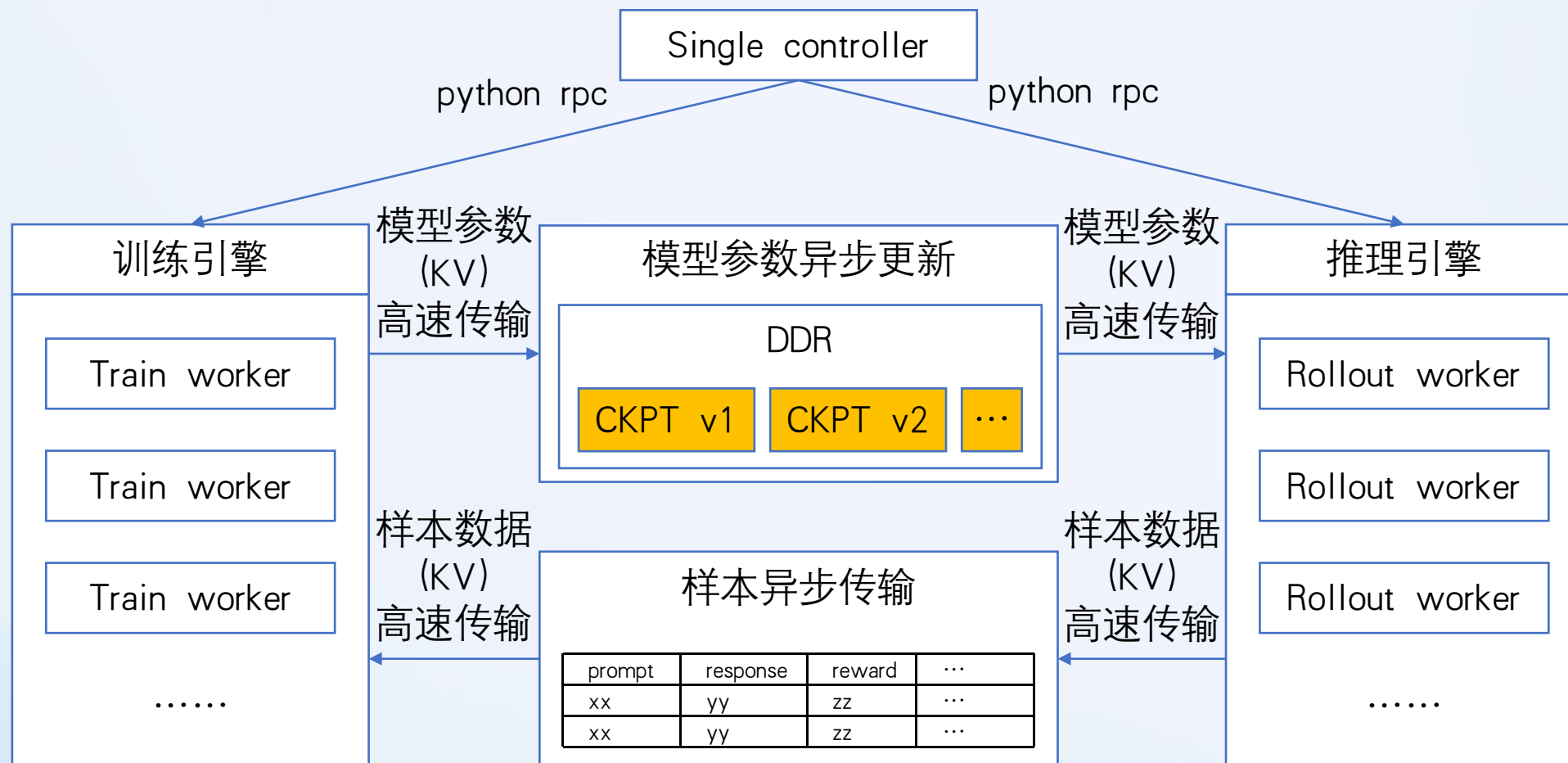
Agentic mode RL采样阶段存在长尾推理请求



训推共卡架构导致NPU资源空泡

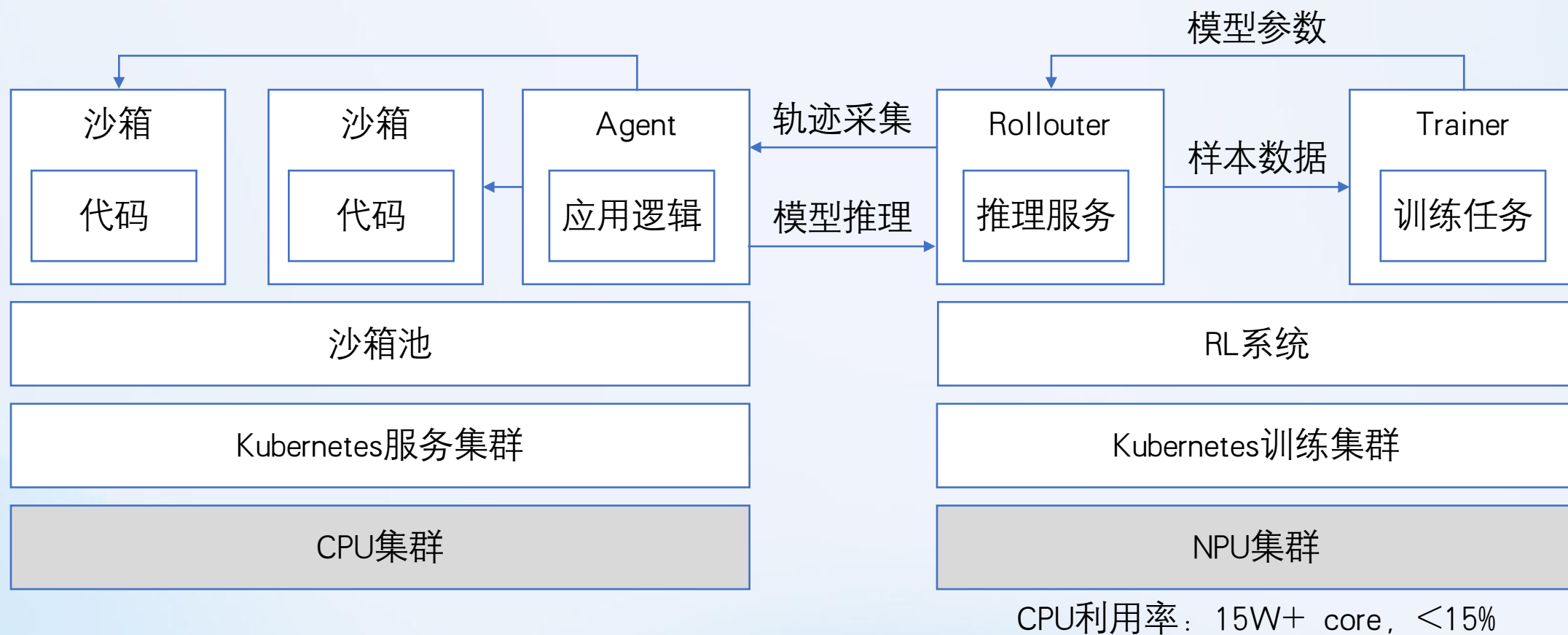


分离式异步训练核心功能：模型参数异步更新&样本异步传输



RL系统的挑战： Agentic RL的训推+环境的跨集群协同问题

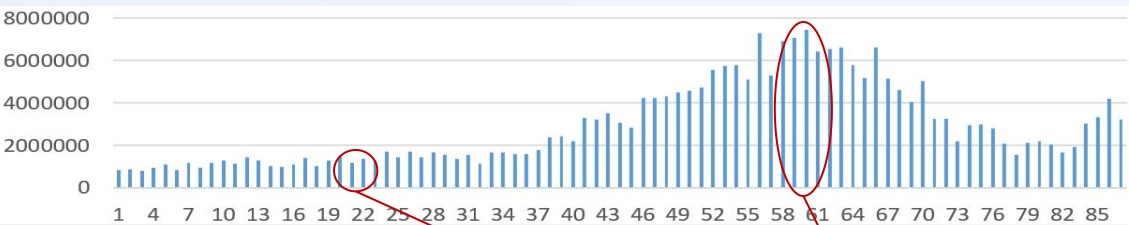
由于训练集群缺乏沙箱能力， Agentic RL需要另外部署一个集群，带来复杂度和资源效率问题





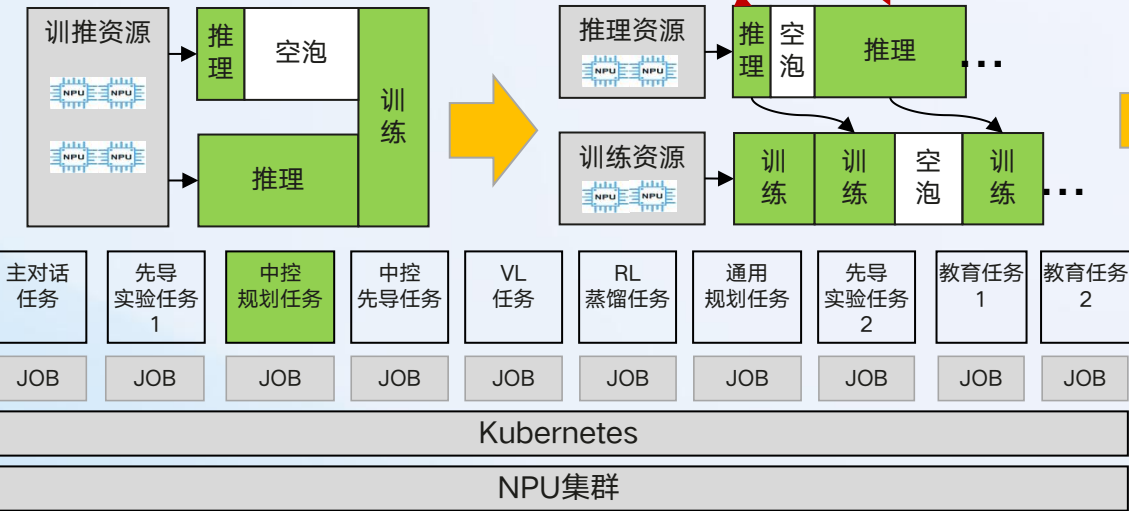
RL系统的挑战：Agentic RL模型实例负载不均衡问题，需要多任务动态调度共享资源空泡

推理输出token个数

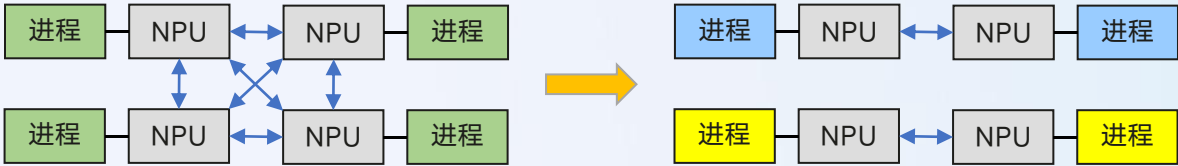


训练轮次

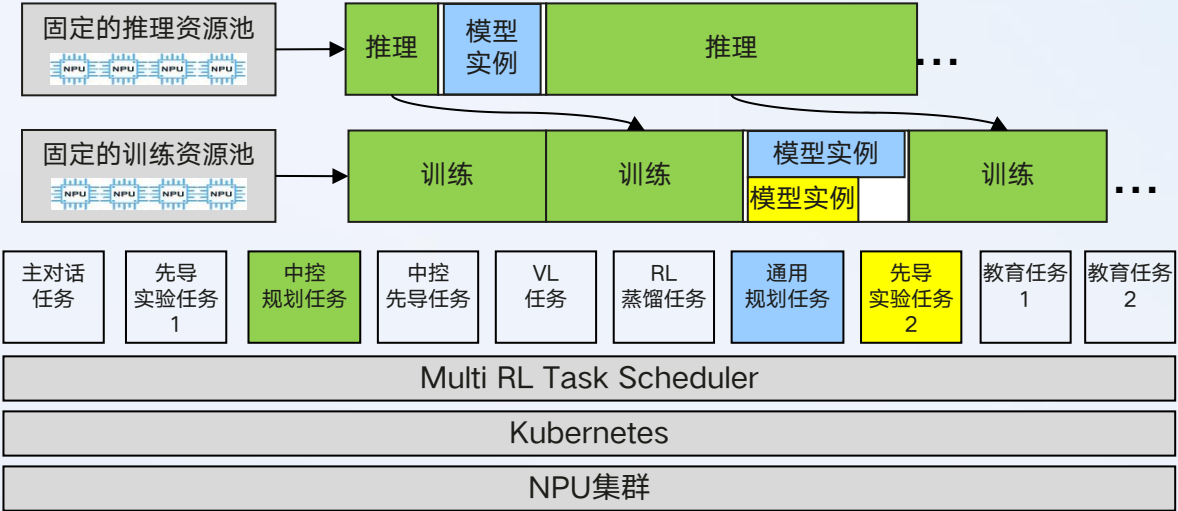
通过训推分离异步可以缓解长尾样本导致的资源空泡，但无法彻底消除空泡



多卡资源成为基础调度单元，集合通信动态重组+模型状态快速切换



动态调度其它RL任务的模型实例填充资源空泡，提升集群资源利用率

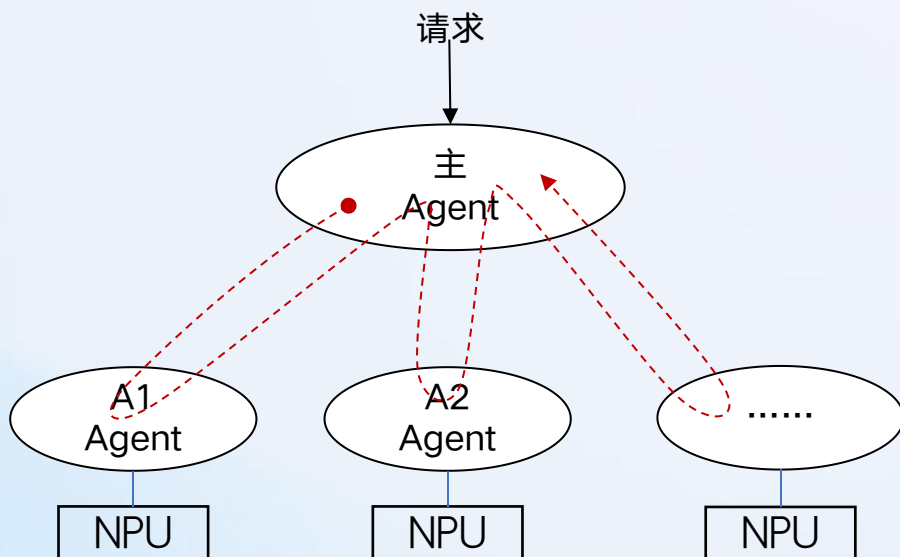


RL系统的挑战: Multi Agent RL下的资源利用率问题

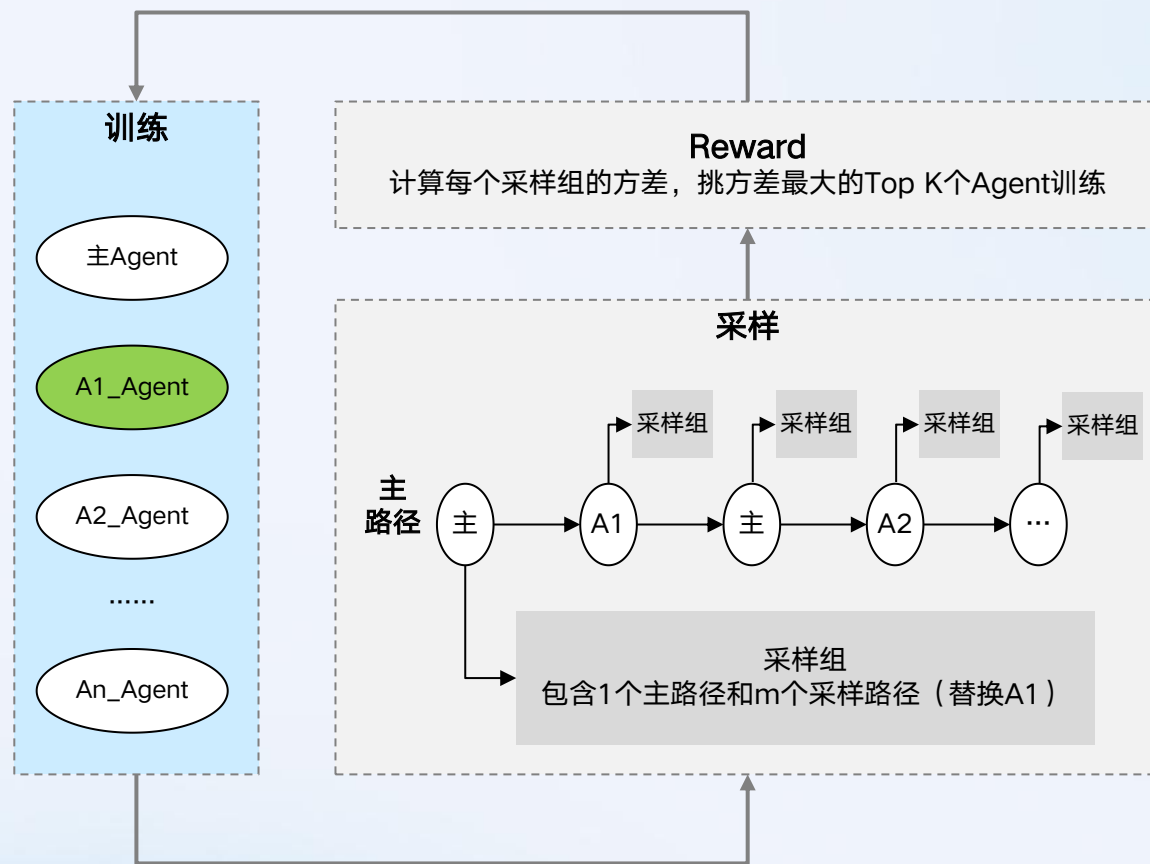
Multiple Agent RL场景, 训练阶段只选取Top K个Agent进行训练, 其它Agent空闲等待, 导致集群资源利用率低

路径生成过程

- 主Agent: 调用子Agent; 调用工具; 返回结果
- 子Agent: 调用工具; 返回结果给主Agent

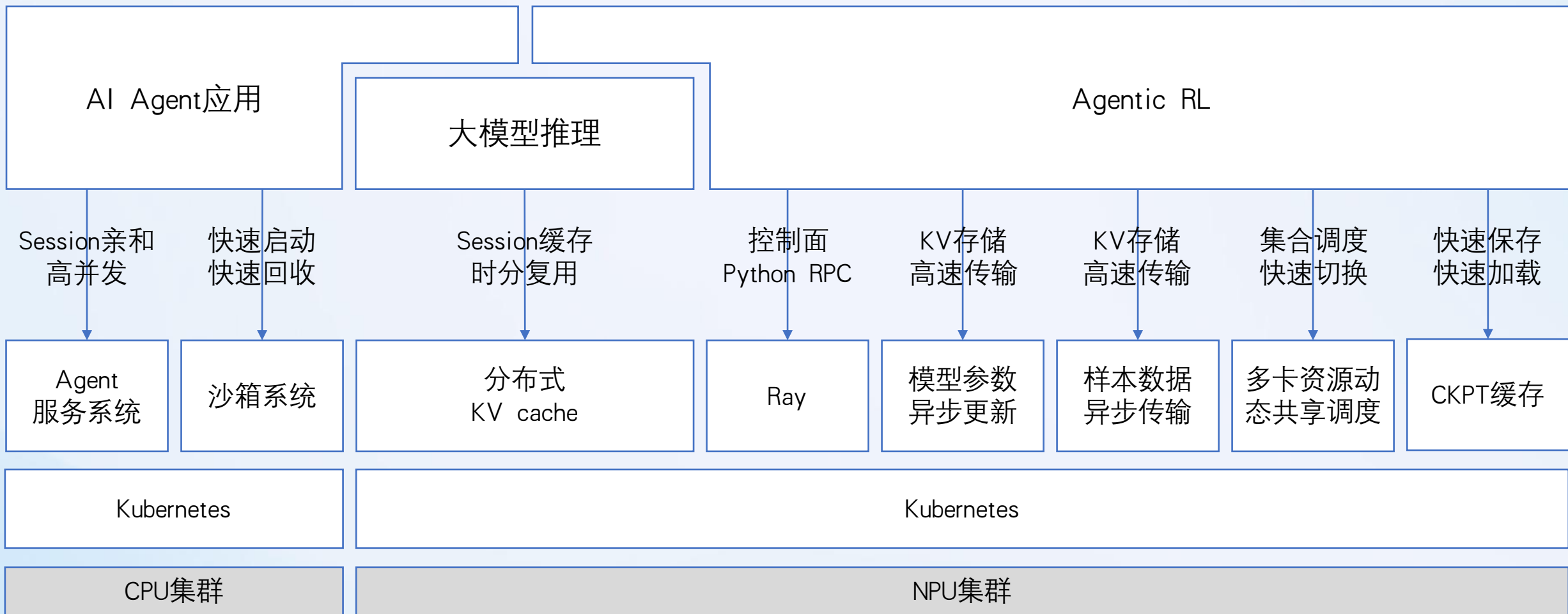


Agent RL训练流程





总结： Agentic AI多样负载带来的需求容易形成烟囱式架构，带来复杂度和资源效率问题



目录

01

Agentic AI负载对分布式系统带来的变化和挑战

02

openYuanrong Serverless分布式计算引擎简介

03

openYuanrong面向Agentic AI负载的关键技术实践

openYuanrong Serverless分布式计算引擎发展历程

2019~2020

2021~2023

2024~Now

FaaS

函数计算底座

事件触发

无状态、短时运行应用、不支持互调

有状态函数

通用分布式计算底座

从FaaS到微服务/大数据/HPC

有状态、长时运行、函数互调、极致弹性

Serverless异构计算

Serverless AI底座

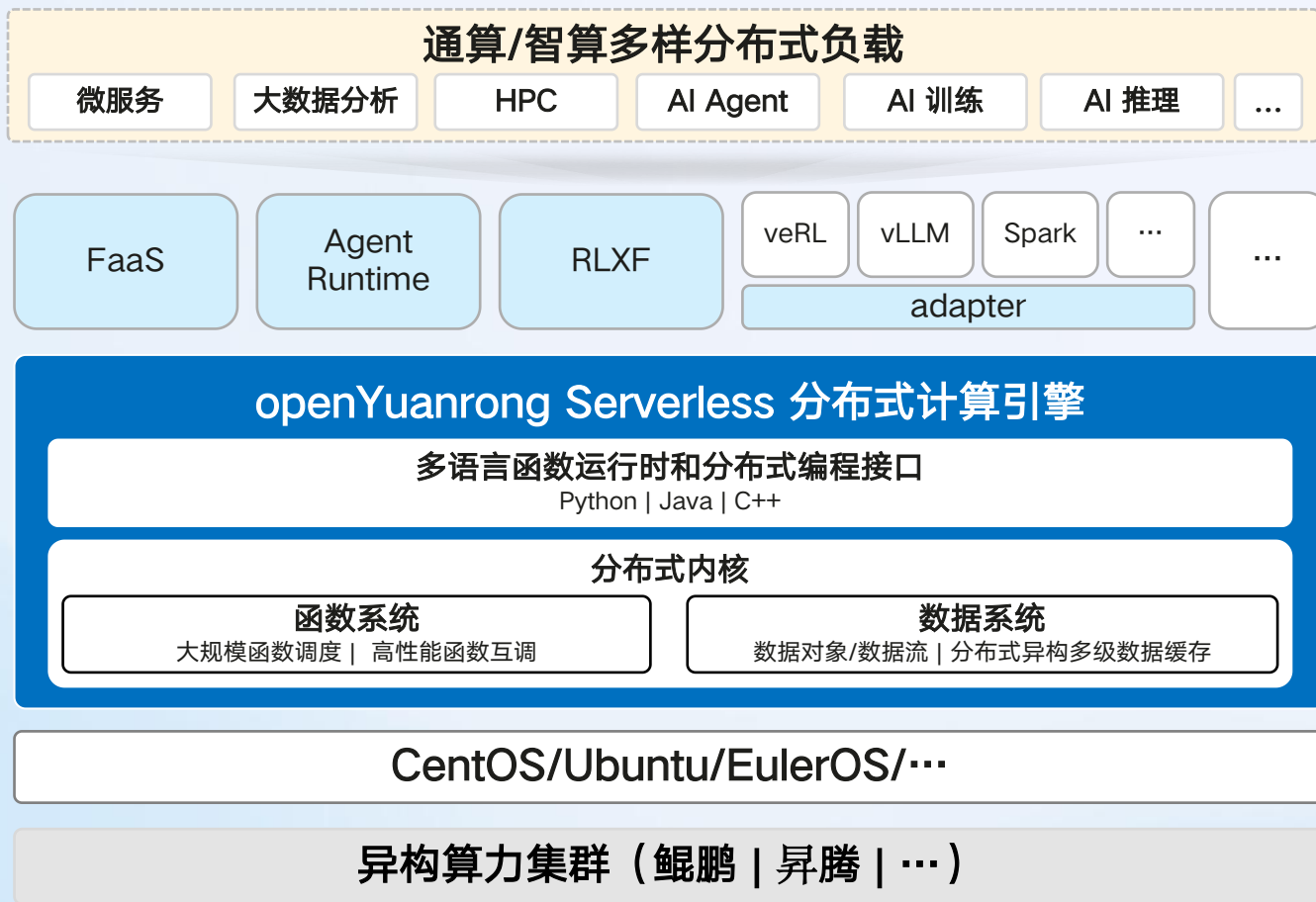
从通算到AI训练/推理/强化学习/Agent

原生支持NPU/灵衢等异构硬件

YuanRong: A Production General-purpose Serverless System for Distributed Applications in the Cloud (ACM SIGCOMM '24)

openYuanrong: Serverless分布式计算引擎, 单机体验编程, 极致分布式运行性能

openYuanrong致力于以一套统一的分布式内核架构支持各类通算/智算分布式应用, 屏蔽集群底层硬件复杂互联和分布式处理细节, 充分发挥集群算力优势, 实现单机体验编程、极致分布式运行性能



分布式内核抽象

单机OS抽象

函数	←→	进程
函数状态	←→	进程私有内存
函数调用	←→	IPC
数据对象/流	←→	进程间共享内存

核心功能

1. 类单机的分布式编程, 支持python/c++/java多语言, 部分接口提供ray的兼容适配层
2. 分布式异构数据系统, 可用于kv cache、ckpt缓存、模型参数更新等场景
3. CPU/NPU资源极速弹性伸缩, LLM秒级扩容
4. 多租隔离



多语言函数运行时：多语言单机体验分布式编程

方式1：原生函数编程

```
public JsonObject process(String request, Context
context) {
    // 省略业务逻辑若干行...
    // 库存管理函数调用另一出库函数“outbound”
    Function func = new Function(context,
“outbound”);
    // 隐藏分布式调度、注册发现、弹性、LB、RPC通信
    ObjectRef<JsonObject> ref =
func.invoke(request);
    JsonObject resp = ref.get(); // 获取调用的结果

    return resp;
}
```

方式2：少量修改自动分布式并行

(无状态)

```
@yuanrong.invoke
def mc_simulation(spot):
    # 省略若干行单次蒙特卡洛模拟逻辑
    return ...

if __name__ == "__main__":
    n = 100
    s = ...

    # invoke快速返回异步future
    # for循环隐藏n次分布式并行函数调度执行
    futures = [mc_simulation.invoke(s) for
i in range(0, n)]
    # 同步阻塞获取n个并行处理结果
    outputs = yuanrong.get(futures)
```

(有状态)

```
@yr.instance
class Bill():
    def __init__(self, customerName)
        self.customerName = customerName
        self.totalMoney = 0

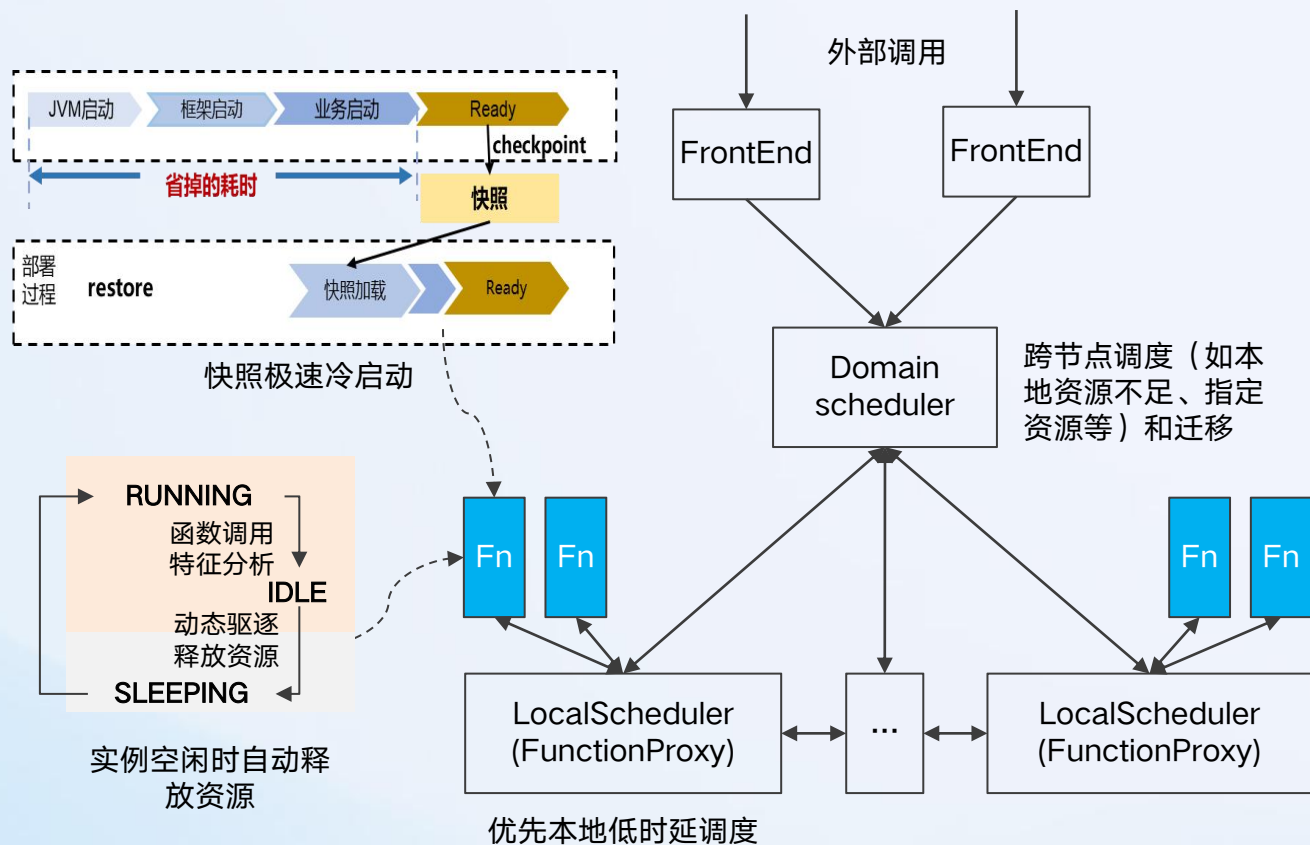
    def add(self, money):
        self.totalMoney = self.totalMoney + money
        return self.totalMoney

    def get(self):
        return self.totalMoney

# 调用“记账”应用
instance = Bill.invoke("Jimmy")
instance.add.invoke(1234)
ret = instance.get.invoke();
# 打印出1234
print(yr.get(ret))
```




函数系统：大规模分布式动态函数调度

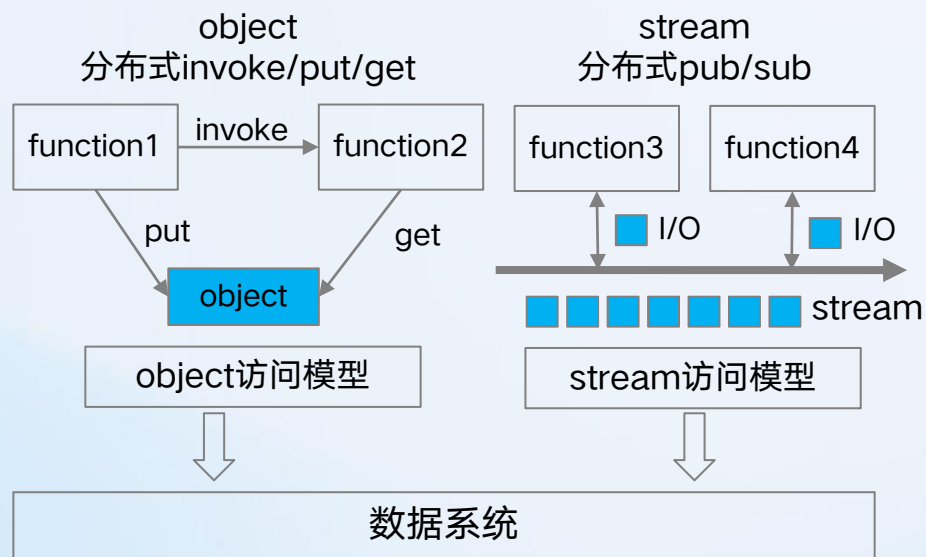


- **分布式分级调度：**分级可扩展，避免中心单点调度瓶颈，支持大规模分布式集群细粒度函数调度
- **动态生命周期管理：**支持运行中动态创建/删除实例，支持自动休眠、唤醒、迁移
- **高效资源利用：**快照极速冷启动，自动水平/垂直弹性和跨节点迁移充分利用各节点资源
- **实例间支持原生互调：**根据实例ID互调，地址无关（不暴露IP），迁移友好，原生支持分布式异步调用

数据系统：近计算高性能分布式内存数据多级缓存

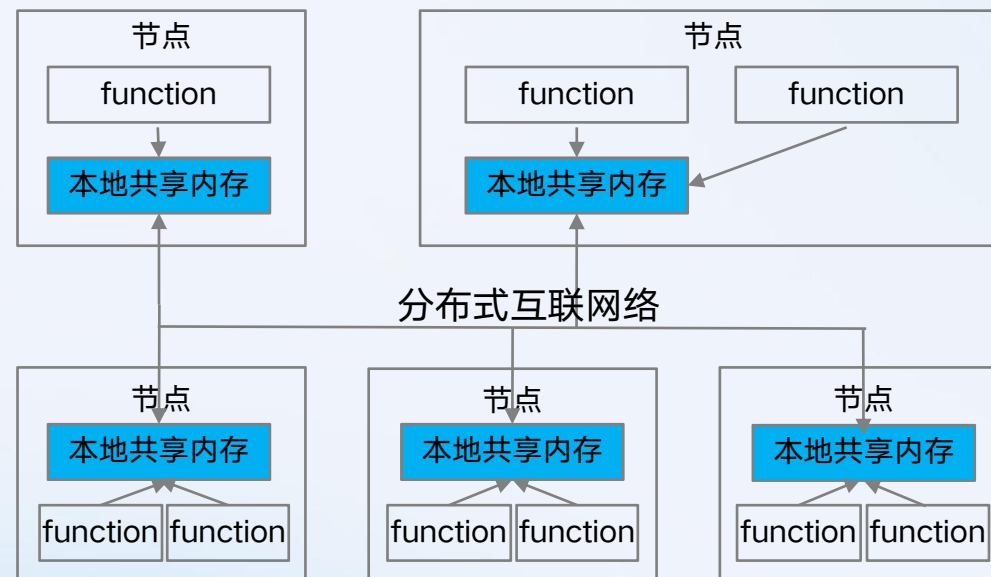
多语义分布式数据访问

- Object用于共享访问同一对象数据，支持Future、自动引用计数、可变、多副本
- Stream传递连续无界数据流，解耦多实例



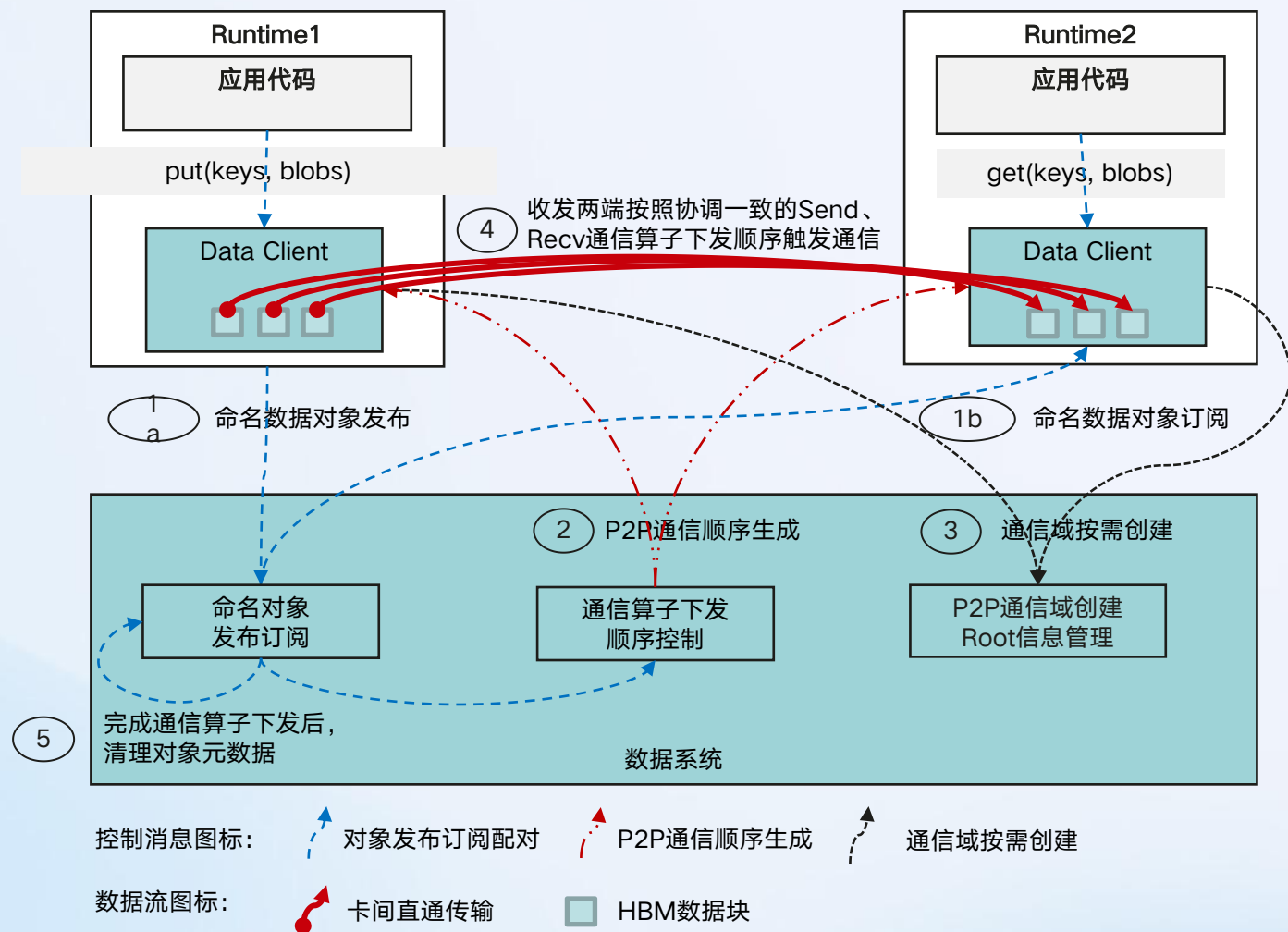
近计算分布式内存共享

- 同节点访问支持本地共享内存免拷贝读写
- 跨节点自动同步用户无感，再次访问走本地共享内存





数据系统：基于异构内存对象实现点对点数据异步高速传输



技术方案

- 扩展分布式数据对象功能，支持异构内存对象如HBM
- 通过分布式数据对象的读写表达点对点异步传输语义
- 对于生产者和消费者都就绪的数据对象，数据系统全局排序下发通信算子，解决死锁和并发问题

目录

01

Agentic AI负载对分布式系统带来的变化和挑战

02

openYuanrong Serverless分布式计算引擎简介

03

openYuanrong面向Agentic AI负载的关键技术实践

3.1

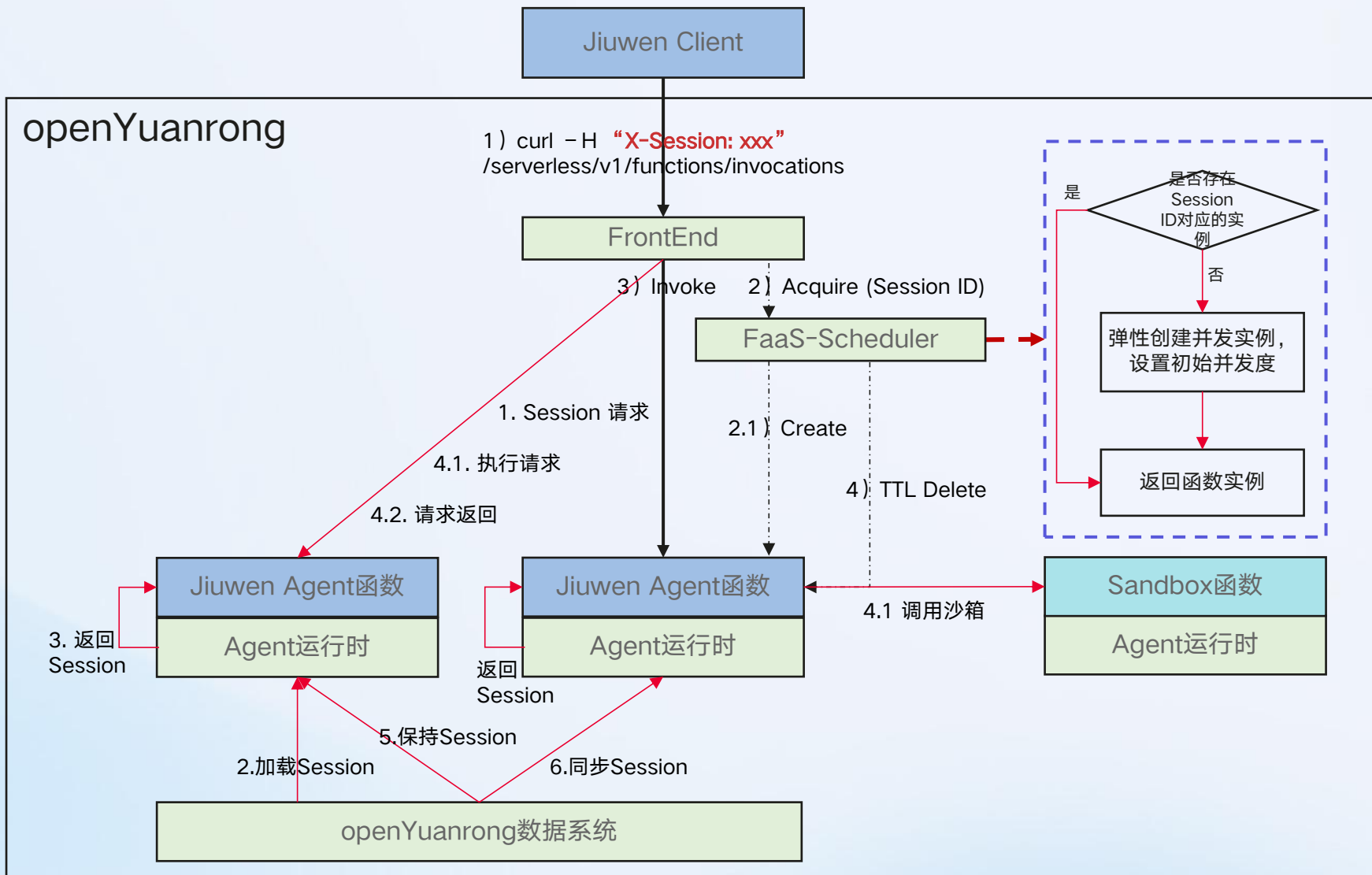
Agent技术实践

3.1

Agentic RL技术实践



基于有状态函数的分布式Agent Runtime方案

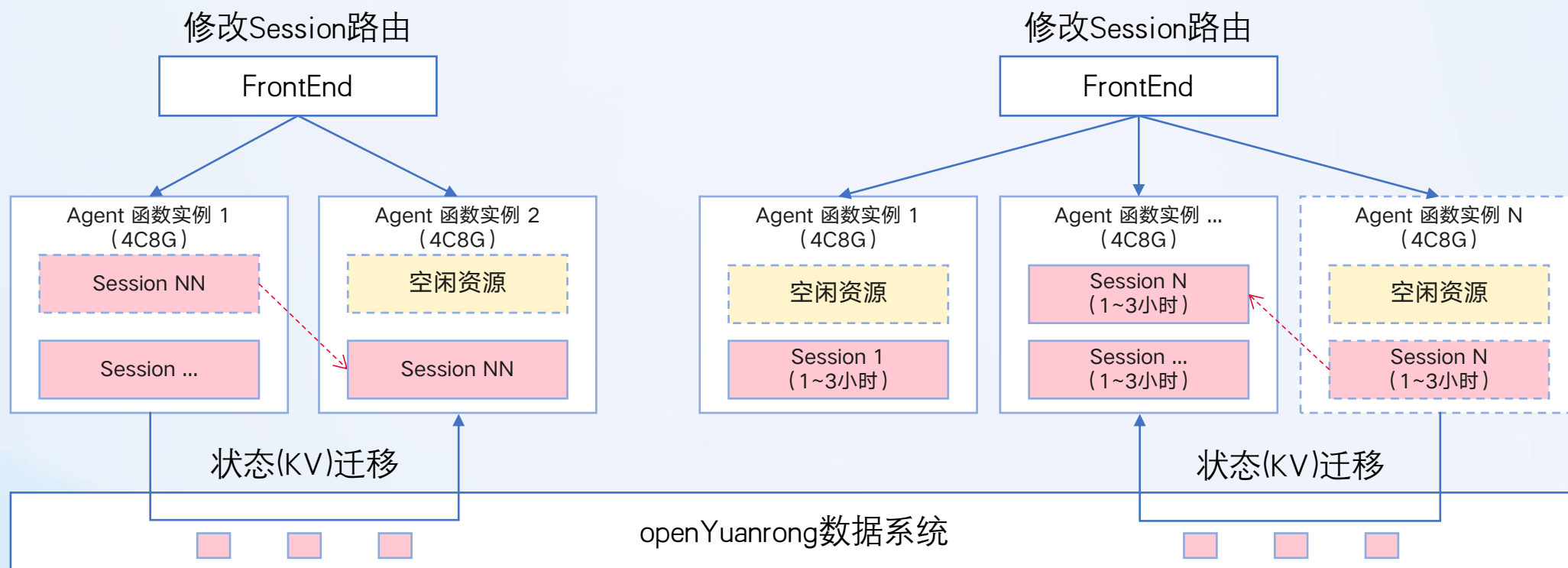


```
public String handler(String event, Context context)
{
    .....
    session = context.getSession();
    List<String> histories = session.getHistories();
    .....
    EventPayload payload = parseEvent(event);
    ....
    histories.add(payload.getEntry());
    ....
}
```

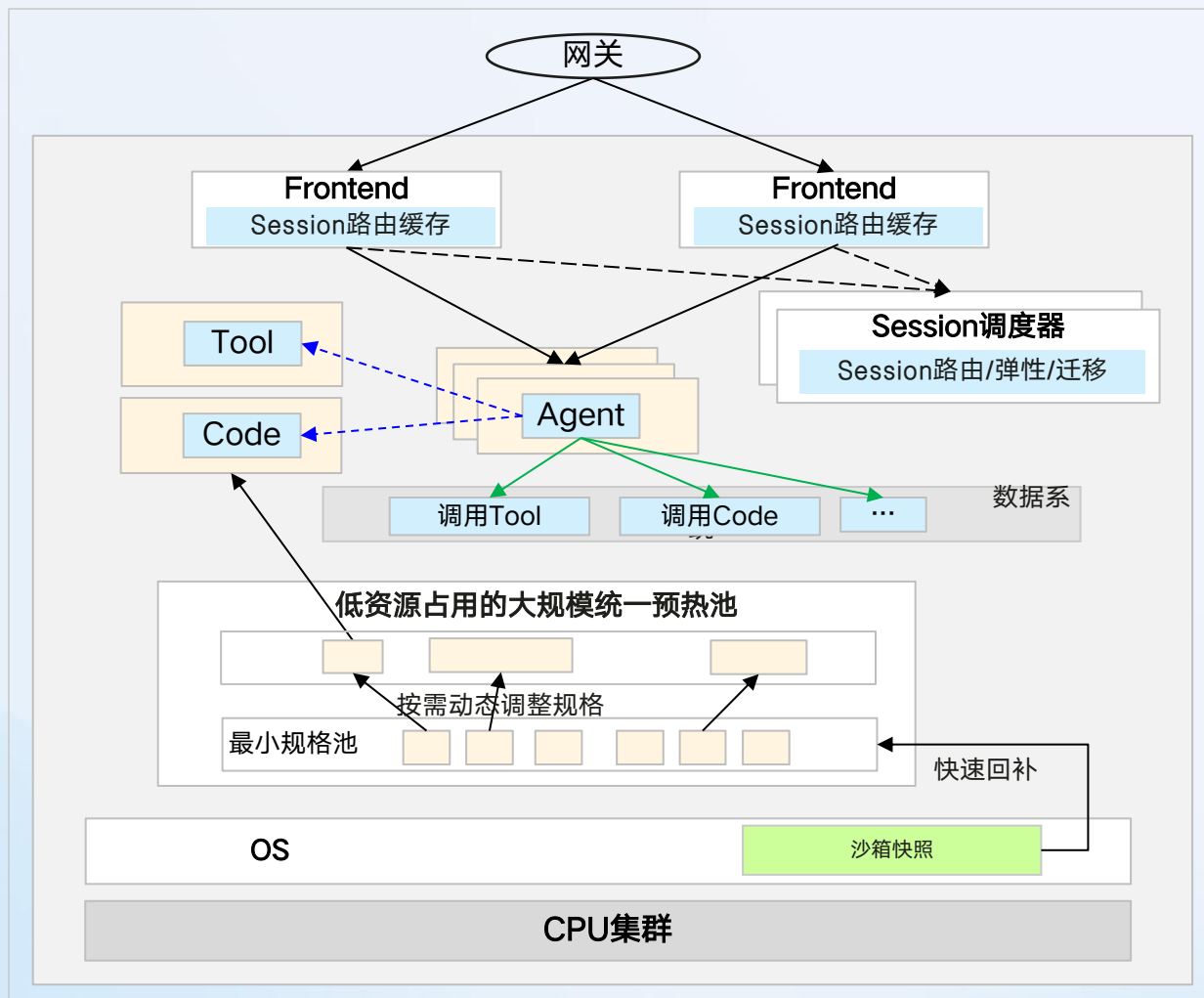
Session动态迁移提升集群负载均衡和资源利用率

负载均衡

迁移长时任务回收空闲实例，提升资源利用率



大规模session-沙箱调度

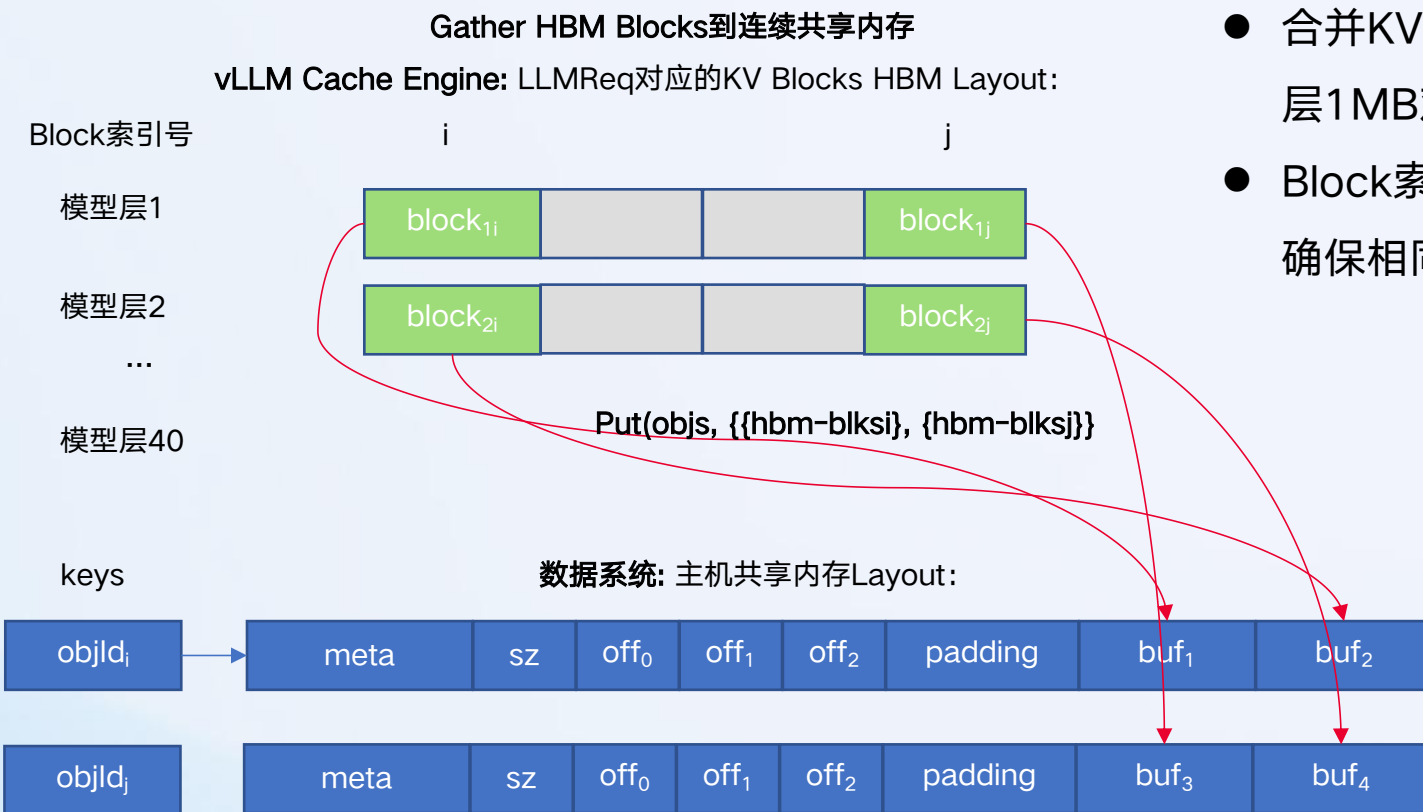


技术方案

- **沙箱超售:** 按最小资源规格预热多个沙箱，在分配任务运行后垂直调整资源规格
- **沙箱快速回补:** 基于镜像快照，秒级回补预热的沙箱实例
- **Session并行调度:** 分离session路由和session资源调度，基于一致性hash实现多调度器扩展

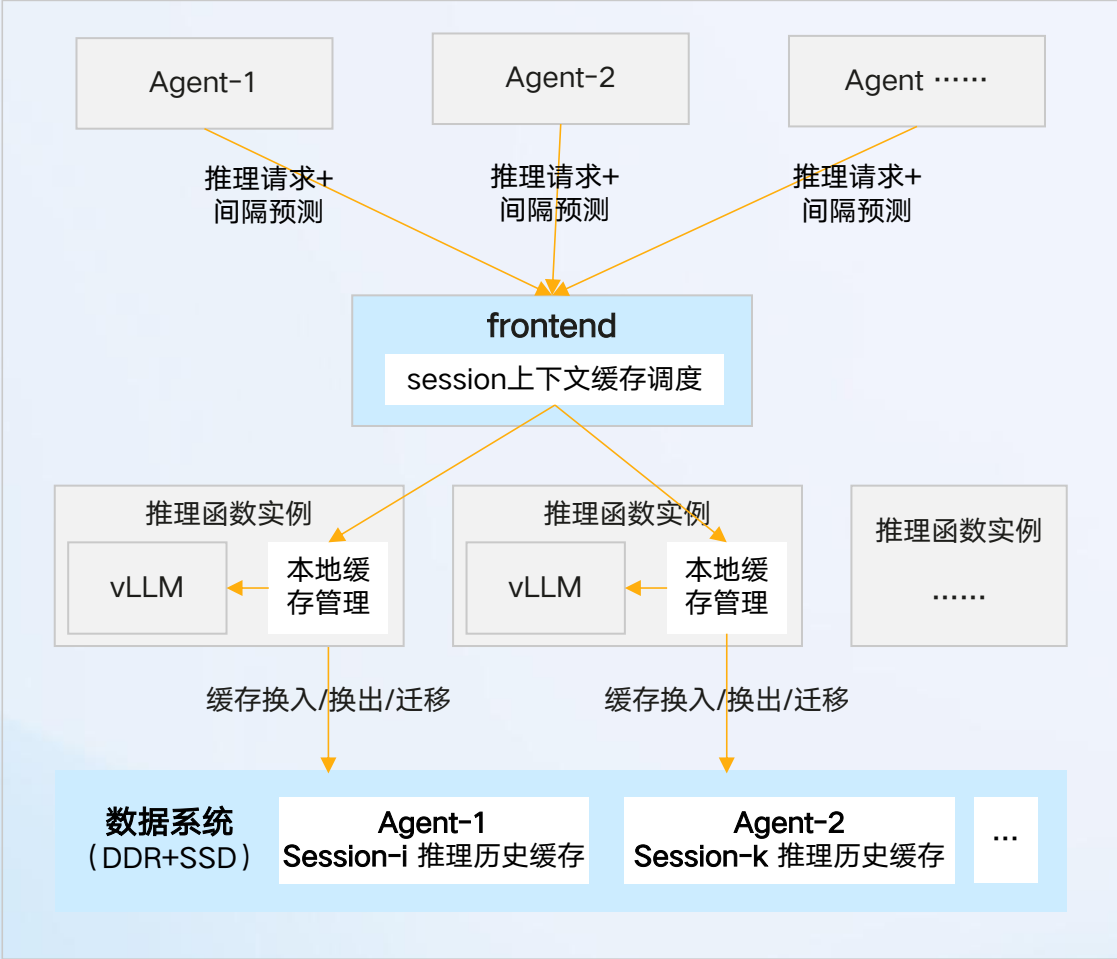
数据系统：基于内存对象聚合管理KV cache

- 合并KV cache减少对象元数据开销，16个32层1MB对象put/get元数据开销 < 1ms
- Block索引号相同的KV cache合并为同对象，确保相同前缀的KV cache同时换入换出





推理服务集群缓存资源在Agent会话之间高效时分复用，提升推理集群吞吐20%+



技术方案

- 会话级推理历史缓存管理：面向Agent会话管理多轮请求的历史缓存，结合高速网络实现异构多级缓存和高速迁移
- Session上下文缓存调度：基于Agent多轮请求间隔预测，提前控制缓存的换入/换出，提前迁移缓存确保负载均衡

难点：预测工具调用时长
下图是实测分布（s）

	P50	P90	p95	P99	Mean
swebench	0.06	0.37	0.53	1.76	0.17
swerebench	0.06	0.3	0.61	3.2	0.2
swebench (O)	0.5	1.12	1.12	4.65	0.5
Science	0.6	4.7	11.2	38	2.37
BFCL	0.6	0.6	1.1	2.9	0.5

目录

01

Agentic AI负载对分布式系统带来的变化和挑战

02

openYuanrong Serverless分布式计算引擎简介

03

openYuanrong面向Agentic AI负载的关键技术实践

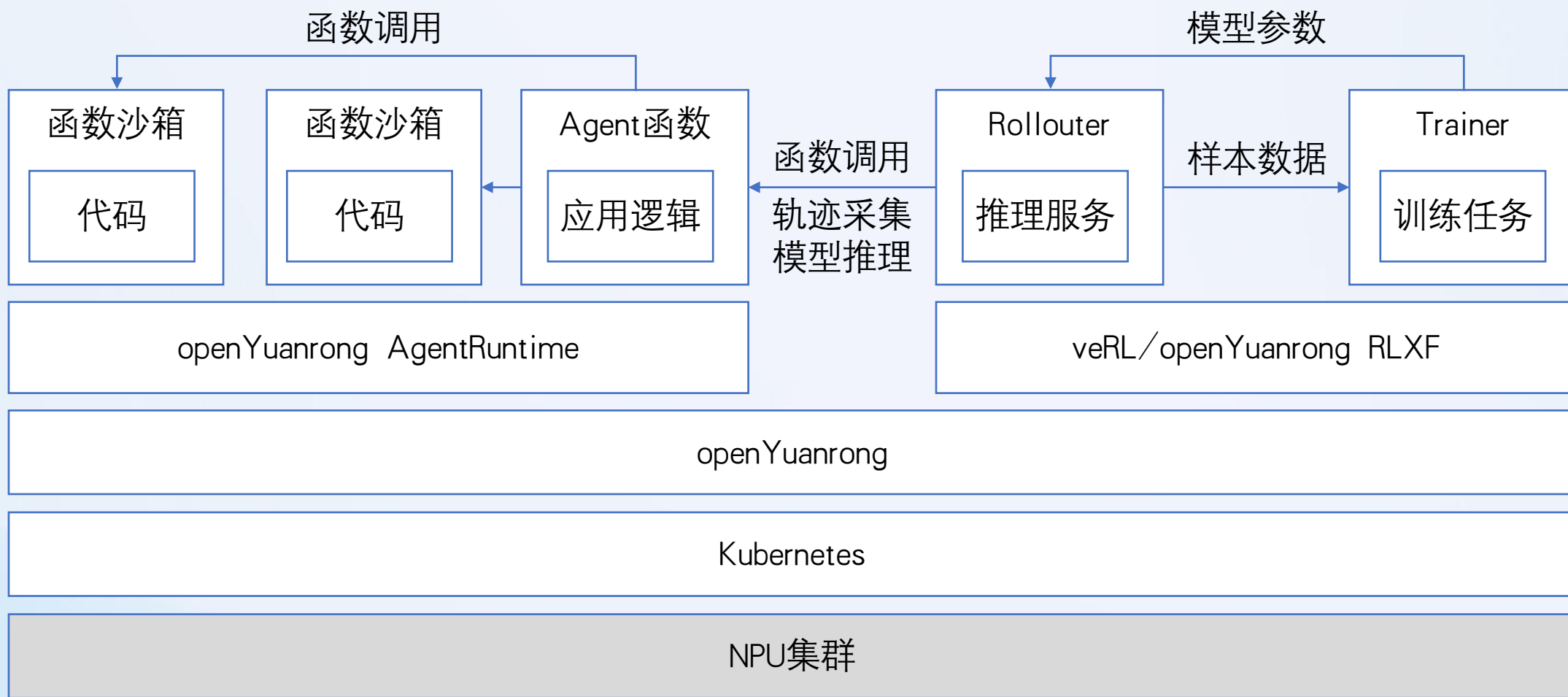
3.1

Agent技术实践

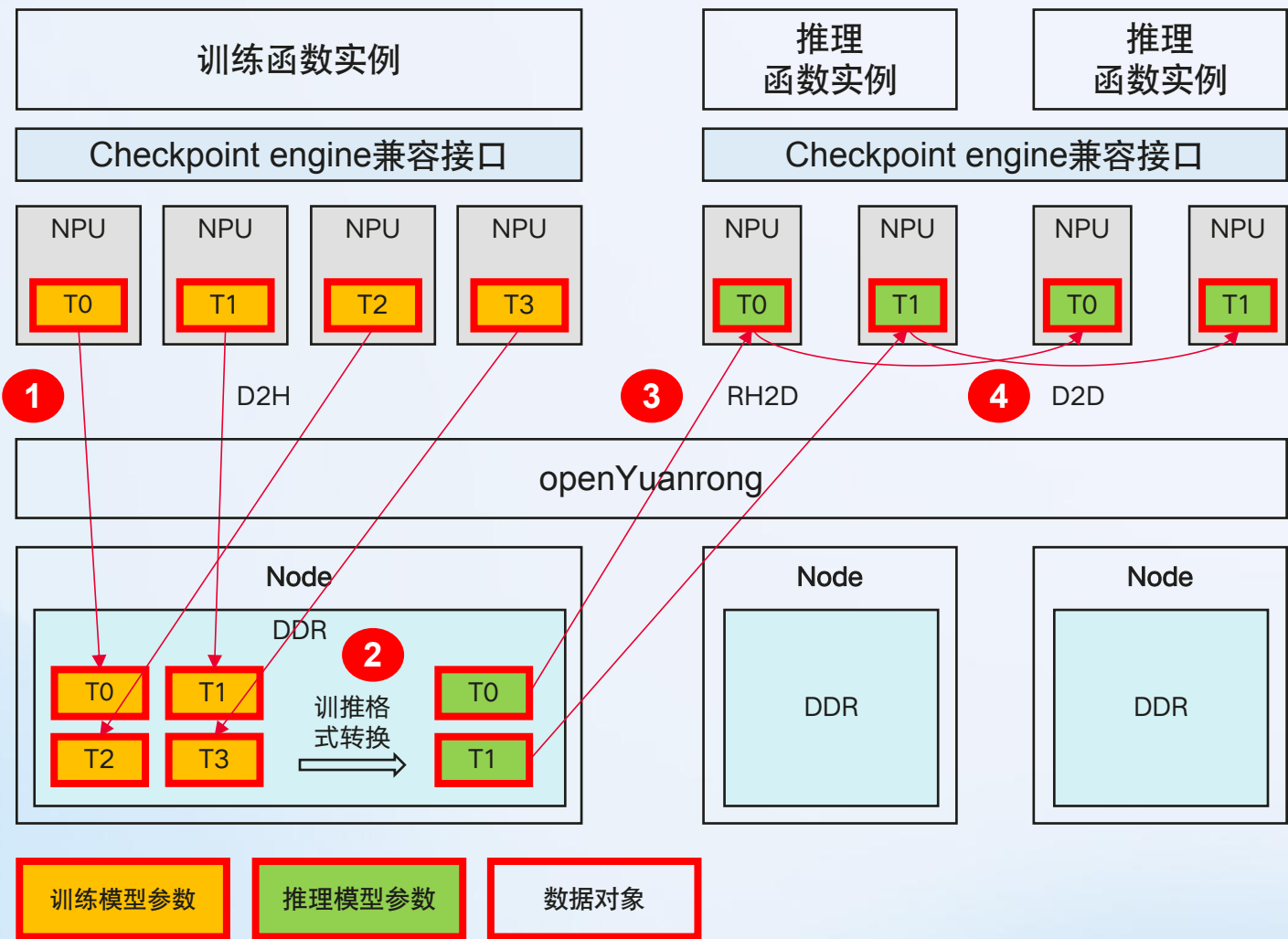
3.1

Agentic RL技术实践

基于openYuanrong单集群实现Agentic RL示例 (CodeAgent)



基于openYuanrong实现模型参数异步更新



技术方案

- 将需要更新的模型参数从训练任务的HBM卸载到DDR，每个参数分片对应一个数据对象
- 利用CPU实现模型参数训推格式的分布式并行转换
- 推理函数实例使用模型参数及分片编号作为对象名称，从远程DDR获取对象至本地HBM
- 推理实例可从其他推理实例的HBM中获取对象至其本地HBM

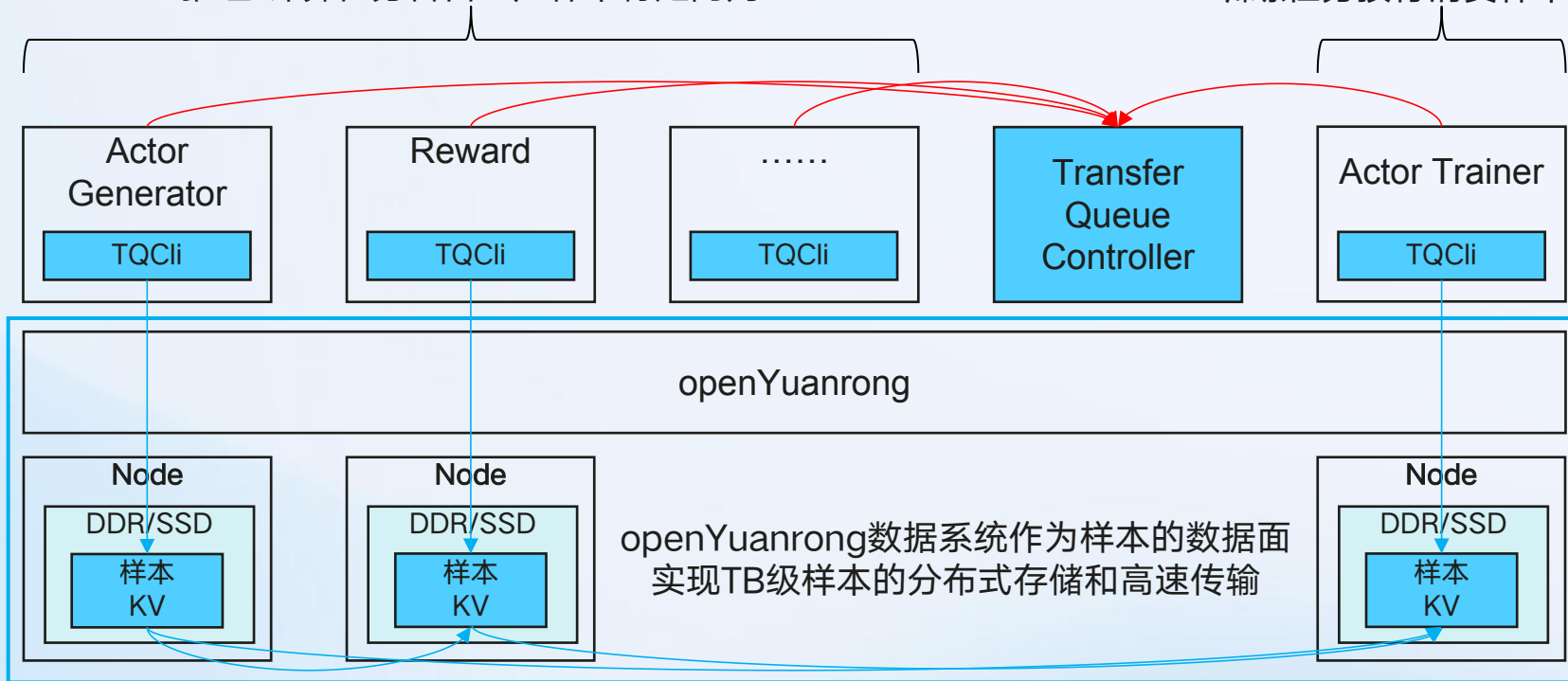
openYuanrong作为TransferQueue数据面后端，实现RL样本数据分布式KV存储和高速传输

强化学习样本表格示例

policy_version: int	sample_id: string	is_processing: bool	prompt: str	response: str	reward: float	next_agent_id: int	prompt_status: bool	response_status: bool	reward_status: bool	...
1	1_1_0	False	"You are cute"	"You are right"	None	2	True	True	False	
1	1_1_1	False	"You are cute"	"You are true"	None	2	True	True	False	
1	2_1_0	True	"You are clever"	"You are ..."	None	None	True	False	False	
...										

推理/计算任务各自生产样本特定的列

训练任务按行消费样本



openYuanrong数据系统提供：

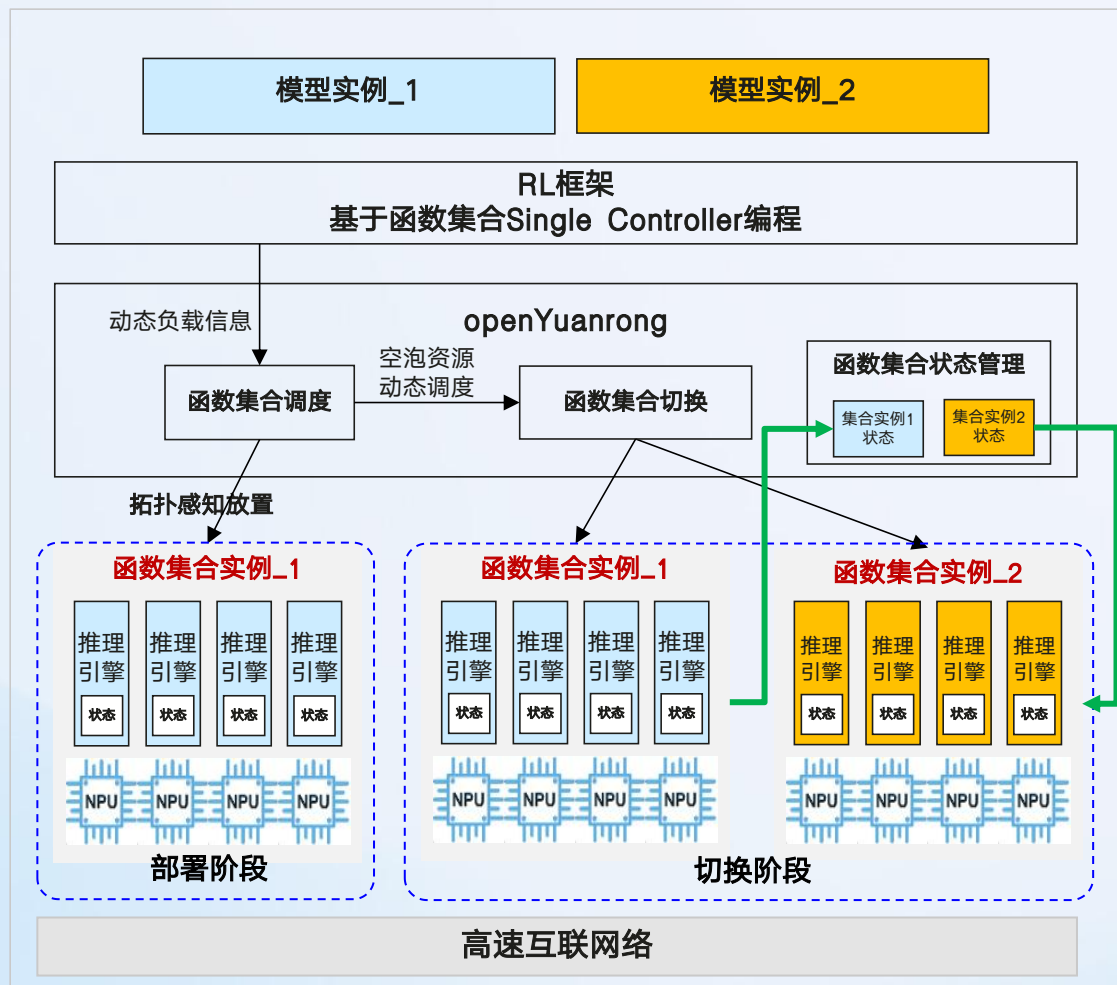
- TQ的KV后端分布式存储能力
- KV跨节点的高速传输能力

TransferQueue相关文档参考：

https://verl.readthedocs.io/en/latest/data/transfer_queue.html

<https://github.com/TransferQueue/TransferQueue>

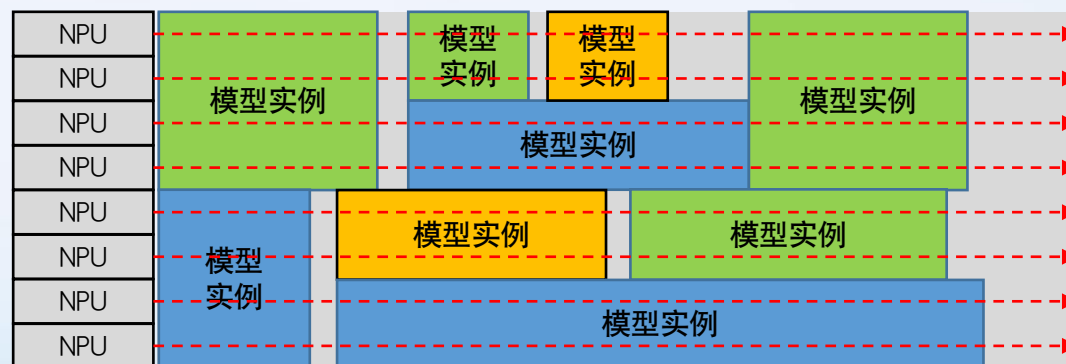
集合调度实现多个RL任务动态共享NPU资源空泡，提升集群NPU利用率10%



技术点

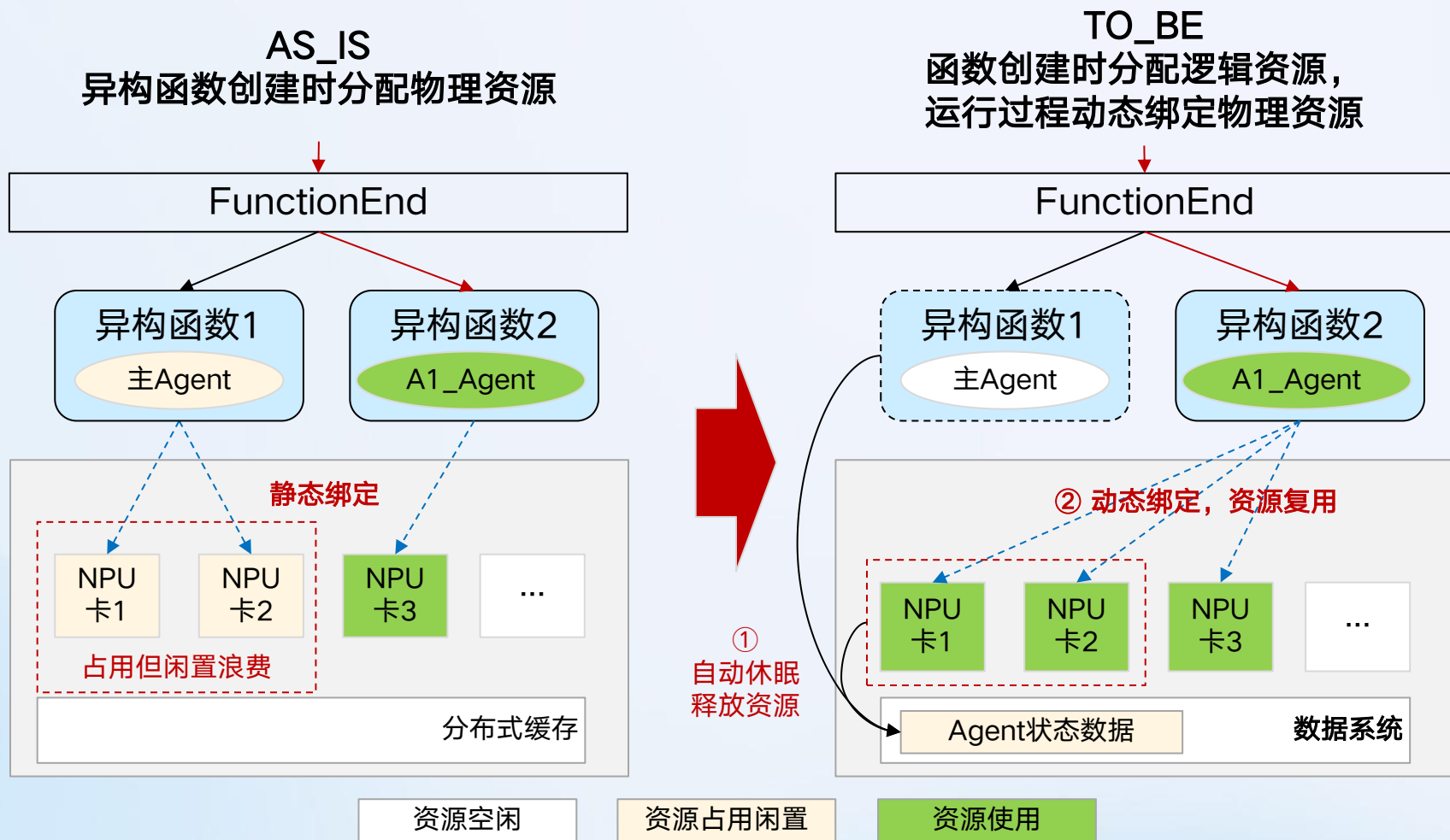
- **函数集合共享资源调度**：结合RL负载识别NPU有效资源空泡并更新全局资源视图，结合**拓扑重调度**其它函数集合的多卡多进程实例来填充空泡
- **函数集合状态极速切换**：多卡多进程状态一致性换入换出，基于**HBM-DDR**的高速网络将切换速度从秒级优化到百毫秒，使能NPU资源高效共享

集合调度效果示意图





异构函数自适应动态调度，提升multi agent场景异构资源利用率10%



技术方案：

- 异构函数实例动态调度：在实例初次调度和再次唤醒时，根据各节点动态更新的实时异构资源视图，动态选取合适的节点和NPU卡运行实例
- 异构函数自动休眠和唤醒：当异构函数实例空闲时，利用数据系统实现快速休眠，释放算力资源；当有业务请求时快速唤醒，并充分利用空闲资源加速计算

openYuanrong支持一行代码平替Ray核心接口

修改方法:

.....

import ray

import ray_adaptor as ray

.....

remote 示例

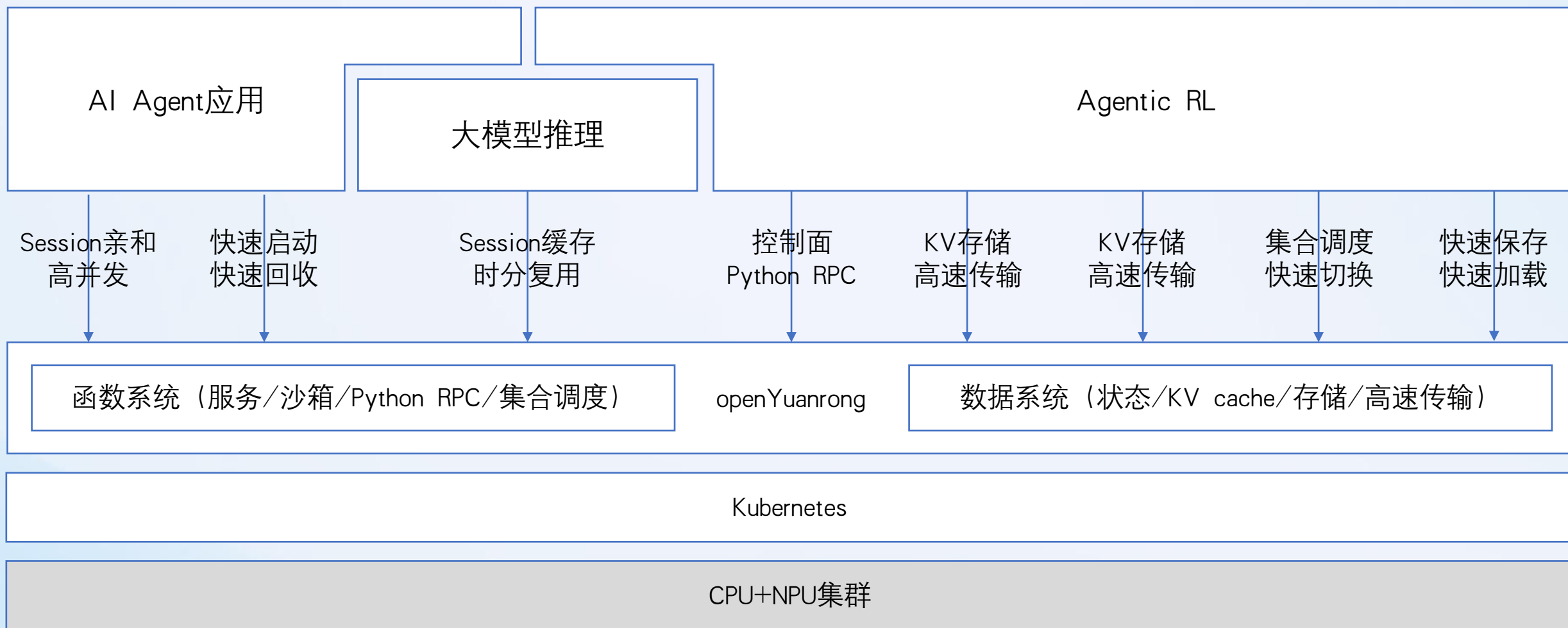
```
import ray_adapter as ray
ray.init()
@ray.remote(num_cpus=1, max_retries=3)
def test_function():
    return "Hello!"
remote_function = test_function.remote()
result = ray.get(remote_function)
print(result)
ray.shutdown()
```

更多示例参考:

<https://gitee.com/openeuler/ray-adapter/blob/master/README.md>

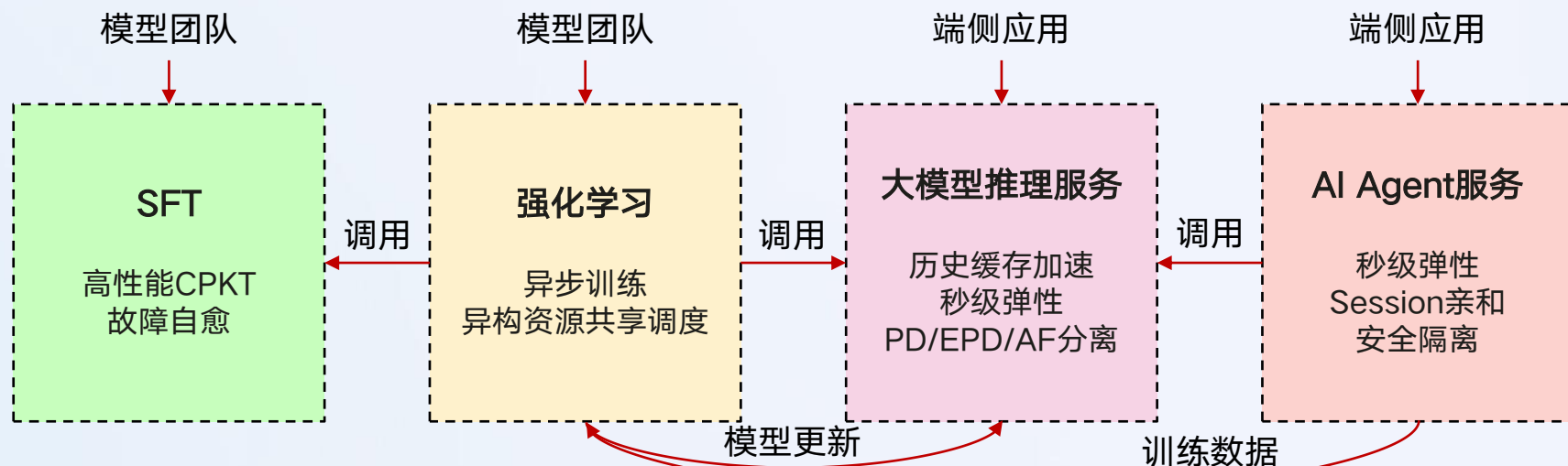


基于openYuanrong积木式架构实现Agentic AI负载的分布式系统





openYuanrong作为统一分布式底座支持华为小艺Agentic AI全场景



小艺Serverless AI平台

分布式训练框架

强化学习框架

分布式推理框架

AI Agent框架

openYuanrong

业务收益:

• Agent

- 单Agent万级session并发
- Agent毫秒级故障恢复
- 分布式KV cache访问性能提升1倍以上
- 大模型推理实例秒级启动

• RL

- 异步训推端到端时延减半
- 集群NPU资源利用率提升10%
- 训推参数同步时延从分钟级缩短至秒级

openYuanrong v0.8.0 正式发布，面向 Agentic AI 全面升级



海报背景为蓝色，顶部有黄色和蓝色的圆点装饰。主标题“openYuanrong v0.8.0 正式发布”采用白色和黄色字体，带有黄色星形装饰。下方有一个白色圆角矩形框，标题为“亮点更新”，内容分为“Agent”部分，列出了两个主要更新点。

openYuanrong v0.8.0 正式发布

亮点更新

Agent

- 新增 Session 抽象，提供 Session 自动生命周期管理和状态高性能分布式共享，结合 Session 亲和调度，实现同一会话内上下文自动持久化，让 Agent 在多轮对话中无需手动维护历史状态。【yuanrong】
- 提供原生分布式 wait/notify API，Agent 可主动挂起等待用户输入并释放资源，收到触发后精准唤醒继续执行，避免 Session 无效忙等，提升 Agent 吞吐。

openYuanrong 开源仓地址



2026年4月13日，openYuanrong 团队正式发布 v0.8.0。新版本为 Agentic AI 场景全面升级，强化对 Agent、强化学习、推理等场景的支持。多语言运行时新增 Session 抽象与 wait/notify 交互原语，支持 Agent 多轮会话与异步交互。调度方面支持安全沙箱的管理和与 Session 弹性迁移，实现实例资源动态聚合。数据系统支持分布式异构多级缓存与 RDMA 协议，提升 KV Cache 跨机访问与样本传输效率。

openYuanrong 开源以来受到了大家的广泛关注和欢迎，并已开始落地互联网、AI、金融等各行业核心场景。感谢每一位参与 openYuanrong 项目贡献的开发者，同时欢迎更多的开发者加入我们！

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

